

BHARATH INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Deemed to be University — Accredited by NAAC)

Faculty of Arts and Science

Department of Computer Science

PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Holistic Smartphone Data Protection System

Integrating Android App Analysis and Secure Metadata Tracking

Submitted by

S.S. Nirmal Raji — Reg No: XXX

S. Lokesh — Reg No: YYY

V. Narendiran — Reg No: ZZZ

Under the Guidance of

Ms. Jaya Sathya

Assistant Professor, Department of Computer Science

APRIL 2025

ACKNOWLEDGEMENT

First and foremost, we offer our sincerest thanks to the Almighty God, whose boundless grace, unwavering protection, and generous blessings sustained us through every challenge and triumph encountered during the course of this project. His guidance has been the invisible force behind every line of code written and every idea developed.

We wish to express our deepest and most heartfelt thanks to our Honorable Founder, Dr. S. Jagathrakshakan, and Dr. J. Sundeep Aanand, President of Bharath Institute of Higher Education and Research, for their visionary leadership and for providing us with world-class academic infrastructure, cutting-edge laboratory facilities, and an inspiring learning environment that made this project a reality.

We extend our profound gratitude to the esteemed Dr. M. Sundararajan, Vice-Chancellor In-charge, for his relentless dedication to academic excellence and his personal encouragement to students pursuing innovative research in the field of computer science and emerging technologies.

We are deeply grateful to Dr. A. Muthukumaravel, Dean PG, Faculty of Arts and Science, and Dr. S. Arumugam, Dean UG, Faculty of Arts and Science, for their administrative support and academic guidance throughout the duration of this project. Their doors were always open to us, and their advice was invaluable.

Special recognition is due to Dr. S. Silvia Priscila, Associate Professor and Head, Department of Computer Science, Faculty of Arts and Science, whose tireless commitment to her students and whose passion for applied research inspired us to pursue a project at the intersection of machine learning and blockchain security. Her encouragement to think beyond conventional solutions shaped the direction of this work.

Our most profound professional gratitude belongs to our beloved project guide, Ms. Jaya Sathya, Assistant Professor, Department of Computer Science. Her expert guidance, patient mentorship, meticulous technical review, and consistent availability for discussion were instrumental in transforming our initial concept into a structured, rigorous, and deployable system. Every meeting with her added clarity to our work and direction to our efforts. We are privileged to have had such an exceptional mentor.

We also extend warm thanks to all faculty members of the Department of Computer Science who supported us formally and informally, sharing their expertise and providing constructive feedback during project demonstrations and seminars. Their collective wisdom enriched every aspect of this documentation and the system it describes.

Finally, we offer our most personal thanks to our families — our parents, siblings, and loved ones — whose unconditional love, patient understanding of our late working hours, and consistent moral support provided the emotional foundation on which this entire effort rested. To our friends and classmates who tested the system, reviewed our documentation, and provided feedback: your contributions, large and small, are woven into every page of this work.

S.S. Nirmal Raji (Reg No: XXX)

S. Lokesh (Reg No: YYY)

V. Narendiran (Reg No: ZZZ)

ABSTRACT

Android smartphones have evolved into the primary computing device for a majority of the global population, serving as the gateway to banking, communication, commerce, healthcare, and personal data management. This extraordinary ubiquity, encompassing over three billion active devices worldwide, has simultaneously rendered the Android ecosystem the single most targeted platform for mobile malware authors and cybercriminals. The security threats posed by malicious Android applications span a broad and continuously evolving spectrum, including trojan horse programs that masquerade as legitimate utilities, ransomware that encrypts user data and demands payment for restoration, spyware that covertly collects credentials and transmits them to remote servers, banking malware that intercepts financial transactions, adware that generates unauthorized revenue through click fraud, and rootkits that establish persistent, privileged access to device resources.

Traditional approaches to Android malware detection, including signature-based antivirus scanning, permission-based heuristic analysis, and rule-based behavioral classification, have demonstrated fundamental and increasingly critical limitations when confronted with the sophistication of contemporary mobile threats. Signature-based systems are inherently reactive, requiring that malware samples be identified, catalogued, and distributed to endpoint systems before protection is activated. This reactive model creates an unavoidable window of vulnerability during which zero-day and newly emerged malware variants operate without detection. Heuristic systems, while more proactive in principle, generate elevated rates of false positives that diminish user trust and introduce operational friction.

This project presents the design, development, and comprehensive evaluation of an AI-driven Android malware detection system, designated MalScanX, that addresses these limitations through the application of advanced machine learning classification and the integration of blockchain technology for model integrity verification and result immutability. The system performs static analysis of Android Package (APK) files, extracting a rich, multi-dimensional feature representation without requiring application execution. Feature categories encompass permission declarations, application component registrations, API call pattern signatures, intent filter configurations, and opcode distribution statistics derived from DEX bytecode analysis.

The extracted feature vectors are classified by a trained XGBoost gradient boosting model, an ensemble learning algorithm renowned for its superior classification accuracy, computational efficiency, resistance to overfitting, and interpretability. The model is trained on a balanced, diverse dataset of labeled benign and malicious applications, and its performance is rigorously evaluated using accuracy, precision, recall, F1-score, and ROC-AUC metrics. The system achieves 97.4% classification accuracy on the held-out evaluation dataset, outperforming baseline methods including Naïve Bayes, Decision Tree, Random Forest, and Support Vector Machine classifiers.

A distinguishing architectural innovation of the MalScanX system is its integration of a JSON-based blockchain mechanism for two critical security functions: tamper-proof storage of the trained detection model, and immutable recording of historical malware detection results. Before every prediction operation, the system verifies that the active model's cryptographic hash matches the hash stored on the blockchain, ensuring that only the authenticated, unmodified model version is used for classification. Detection results are permanently recorded on the blockchain with timestamps and application identifiers, enabling rapid cross-verification of repeat submissions without full re-analysis.

The system architecture supports three distinct user roles — API administrators, application developers, and end users — each with tailored interfaces and functional permissions implemented through a comprehensive role-based access control framework. A graded risk stratification module categorizes detected malware into four severity tiers from Low Risk to Critical Threat. An automated blocker module enforces protective actions including prevention of malicious application installation and blocking of harmful developer submissions. A real-time notification module ensures that all stakeholders receive timely, actionable security alerts.

The MalScanX system represents a significant contribution to the field of mobile security, demonstrating the practical viability of combining artificial intelligence with blockchain verification to create a detection platform that is accurate, trustworthy, transparent, and adaptable to the evolving threat landscape of Android malware.

Keywords: Android Security, Malware Detection, Machine Learning, XGBoost, Static Analysis, APK Analysis, Blockchain Verification, Mobile Security, Artificial Intelligence, Feature Extraction, Role-Based Access Control, MalScanX, Gradient Boosting.

TABLE OF CONTENTS

S.No.	Description	Page No.
	Abstract	vi
	Table of Contents	vii
	List of Figures	ix
	List of Tables	x
1.	INTRODUCTION	
1.1	An Overview	
1.2	Objective and Scope of the Project	
1.3	Literature Survey	
1.4	Problem Statement	
2.	SYSTEM ANALYSIS	
2.1	Existing System	
2.2	Disadvantages of Existing System	
2.3	Proposed System	
2.4	Advantages of Proposed System	
3.	SYSTEM CONFIGURATION	
3.1	Hardware Requirements	
3.2	Software Requirements	
4.	SYSTEM DESIGN	
4.1	Overall Architecture Diagram	
4.2	Methodology / Algorithm Adopted	
5.	SYSTEM IMPLEMENTATION	
5.1	Module 1 — Android Malware App Detector API	
5.2	Module 2 — Blockchain Integration	
5.3	Module 3 — End User Access Control	
5.4	Module 4 — AndroMal Model Build and Train	
5.5	Module 5 — Android App Malware Predictor	

5.6	Module 6 — Malware Level Indicator	
5.7	Module 7 — Blocker	
5.8	Module 8 — Alerts and Notification	
6.	SYSTEM TESTING	
6.1	Unit Testing	
6.2	Integration Testing	
6.3	Black Box Testing	
6.4	White Box Testing	
6.5	Verification Testing	
6.6	Validation Testing	
6.7	User Acceptance Testing	
7.	RESULTS AND DISCUSSION	
8.	CONCLUSION	
9.	LIMITATIONS AND SCOPE OF FUTURE WORK	
10.	REFERENCES	
11.	APPENDIX	
11.1	Source Code	
11.2	Output Screenshots	

LIST OF FIGURES

Fig. No.	Figure Title	Page No.
Fig 1.1	Android Malware Distribution Categories 2022–2025	
Fig 1.2	Malware Detection Evolution Timeline	
Fig 4.1	Overall MalScanX System Architecture Diagram	
Fig 4.2	XGBoost Gradient Boosting Algorithm Flowchart	
Fig 4.3	Static Feature Extraction Pipeline	
Fig 4.4	Blockchain Hash Chain Structure	
Fig 4.5	Data Flow Diagram — Level 0	
Fig 4.6	Data Flow Diagram — Level 1	
Fig 5.1	Module Interaction Diagram	
Fig 5.2	Role-Based Access Control Structure	
Fig 5.3	AndroMal Model Training Pipeline	
Fig 5.4	APK Analysis Workflow	
Fig 6.1	Test Coverage Summary	
Fig 7.1	Classification Accuracy Comparison Chart	
Fig 7.2	Confusion Matrix — XGBoost Model	
Fig 7.3	ROC Curve — MalScanX vs Baseline Models	
Fig 7.4	F1-Score Comparison Bar Chart	
Fig 7.5	Feature Importance Plot	
Fig 7.6	System Response Time Distribution	
Fig 11.1	Admin Login Page Screenshot	
Fig 11.2	Dataset Upload Page Screenshot	
Fig 11.3	Feature Extraction Results Page Screenshot	
Fig 11.4	XGBoost Classification Output Screenshot	
Fig 11.5	Blockchain Block Information Screenshot	
Fig 11.6	User Dashboard and Scan Result Screenshot	

LIST OF TABLES

Table No.	Table Title	Page No.
Table 3.1	Hardware Requirements Specification	
Table 3.2	Software Requirements Specification	
Table 4.1	XGBoost Hyperparameter Configuration	
Table 4.2	Feature Category Description	
Table 5.1	Module Summary and Responsibilities	
Table 6.1	Unit Test Cases	
Table 6.2	Black Box Test Cases	
Table 6.3	Integration Test Scenarios	
Table 6.4	User Acceptance Test Criteria	
Table 7.1	Model Performance Metrics	
Table 7.2	Baseline Comparison Table	
Table 7.3	Blockchain Performance Metrics	
Table 7.4	System Response Time Analysis	
Table 7.5	Feature Importance Rankings	

CHAPTER 1: INTRODUCTION

1.1 An Overview

The rapid proliferation of Android-powered smartphones has fundamentally transformed the nature of personal and professional computing. Android, developed by Google and based on the Linux kernel, now powers more than 72% of all mobile devices globally, representing a user base exceeding three billion active installations as of 2024. This extraordinary market penetration has established Android as the de facto computing platform for populations across all socioeconomic categories and geographic regions, enabling unprecedented access to digital services ranging from mobile banking and e-commerce to telemedicine and educational platforms.

However, this same ubiquity has created an immense and continuously expanding attack surface that is actively exploited by an increasingly sophisticated community of malware authors and cybercriminals. The Android platform's open-source nature, its diverse hardware ecosystem, the fragmentation of Android versions across devices, and the availability of multiple application distribution channels beyond the official Google Play Store collectively create conditions that facilitate the development and distribution of malicious applications at scale.

Android malware represents one of the most serious and rapidly evolving cybersecurity threats in the contemporary digital landscape. Malicious applications distributed through the Android ecosystem employ a wide variety of attack methodologies to achieve their objectives. Banking trojans intercept credentials, one-time passwords, and transaction authorizations to enable unauthorized financial transfers. Spyware applications operate silently in the background, harvesting location data, contact lists, SMS messages, call logs, and application usage patterns for transmission to remote command-and-control servers. Ransomware programs encrypt device storage and demand payment for decryption keys. Adware applications generate fraudulent advertising revenue through simulated user interactions. Rootkit malware establishes persistent privileged access to device resources, enabling long-term covert operations that survive factory resets and application removal attempts.

The challenge of detecting and neutralizing Android malware has intensified dramatically as threat actors have adopted increasingly sophisticated evasion techniques. Code obfuscation transforms readable source code into functionally equivalent but syntactically unrecognizable representations that defeat pattern-based signature matching. Polymorphic malware dynamically modifies its own

code during execution to produce variants with distinct signatures while preserving malicious functionality. Dynamic code loading defers the retrieval and execution of malicious payloads until runtime, hiding them from static analysis systems that examine only the initial application package. Reflection-based method invocation conceals API call patterns from static analyzers by resolving method names programmatically at runtime rather than encoding them statically in the bytecode.

Traditional malware detection technologies, despite decades of development and deployment, remain fundamentally ill-equipped to address these sophisticated evasion mechanisms. Signature-based detection systems maintain databases of cryptographic hashes and code pattern strings derived from known malware samples. While highly reliable for identifying previously catalogued threats, these systems are powerless against novel variants and zero-day exploits that have not yet been analyzed and catalogued. The inevitable lag between malware emergence and signature database update creates windows of vulnerability that are exploited to maximum effect by threat actors who continuously generate new variants.

The application of machine learning to the problem of Android malware detection represents a paradigm shift from reactive signature matching to proactive pattern recognition. Machine learning models learn discriminative representations of application behavior from labeled training data, enabling them to generalize beyond known samples and identify malicious applications based on behavioral similarity to previously encountered threats. This capability is particularly valuable for detecting novel malware variants that share behavioral characteristics with known malware families despite differing at the code signature level.

Static analysis approaches, which examine application code and metadata without executing the application, offer particular advantages for machine learning-based detection. Static analysis can be conducted safely, rapidly, and at scale without the resource overhead of dynamic execution environments. The rich feature information available from APK files — including permission declarations, API usage patterns, bytecode structure, and manifest configurations — provides a comprehensive behavioral fingerprint that enables accurate classification.

This project presents the complete design, implementation, and evaluation of the MalScanX Holistic Smartphone Data Protection System, an AI-driven Android malware detection framework that combines XGBoost-based machine learning classification with blockchain-backed model integrity verification. The system addresses the primary limitations of existing detection approaches while

introducing novel architectural features that enhance both detection accuracy and operational trustworthiness. The integration of blockchain technology for model storage and result recording represents a significant architectural innovation that ensures the integrity of the detection pipeline against both external attacks and insider threats.

The system is designed to serve three distinct stakeholder categories: system administrators responsible for platform management and model maintenance, application developers seeking pre-publication security validation for their submissions, and end users requiring real-time protection during application installation. A comprehensive role-based access control framework ensures that each stakeholder category receives tailored functionality and appropriate access to system capabilities. The Bootstrap-based responsive web interface ensures accessibility across diverse device types and technical backgrounds.

The technical implementation leverages Python 3.8 as the primary development language, supported by Flask for API management, XGBoost for classification, Pandas and NumPy for data processing, Scikit-learn for model evaluation, Matplotlib and Seaborn for visualization, MySQL for persistent data storage, and a custom JSON-based blockchain module for verification. The system is designed for deployment on Windows or Linux server environments with the hardware specifications detailed in Chapter 3.

1.2 Objective and Scope of the Project

Aim of the Project

The central aim of this project is to design and develop a robust, intelligent, and verifiable Android malware detection system that leverages the combined power of gradient boosting machine learning and blockchain-based integrity verification to provide accurate, trustworthy, and real-time protection for Android device users, application developers, and platform administrators. The system aims to overcome the fundamental limitations of conventional detection approaches while delivering a scalable architecture suitable for real-world deployment.

Specific Objectives

- To develop a complete end-to-end Android malware detection system capable of identifying and preventing the distribution and installation of malicious applications through automated, intelligent analysis.
- To collect, curate, preprocess, and balance comprehensive datasets of both benign and malicious Android applications to support robust supervised machine learning model training.
- To design and implement a static feature extraction engine that systematically derives permission declarations, API call patterns, application component structures, intent filter configurations, and opcode distribution features from Android APK files.
- To train, tune, and validate an XGBoost gradient boosting classification model that achieves high accuracy, precision, recall, and F1-score performance on diverse test datasets.
- To integrate a JSON-based blockchain mechanism for tamper-proof storage of the trained detection model and immutable recording of historical malware detection results.
- To implement a cross-verification system that authenticates detection predictions by comparing active model hashes against blockchain-stored reference values before every prediction operation.
- To design a multi-tier risk stratification module that translates classification confidence scores into actionable risk levels communicable to non-technical users.
- To build an automated blocker module that prevents the installation or publication of applications classified at High Risk or Critical Threat levels.
- To implement a real-time notification system that delivers timely security alerts to users, developers, and administrators upon detection of harmful applications.
- To create a role-based access control architecture supporting API administrators, application developers, and end users with appropriate permission boundaries and tailored interfaces.
- To evaluate system performance comprehensively using standard classification metrics, comparative baseline analysis, and operational performance benchmarking.

Scope of the Project

The project scope encompasses the complete lifecycle of the MalScanX system from requirements specification through design, implementation, testing, and performance evaluation. The system is scoped to perform static analysis exclusively, examining APK file structure and metadata without

requiring application execution in an instrumented runtime environment. This scope boundary ensures analysis safety, computational efficiency, and practical deployability. The blockchain integration is implemented as a JSON-based module appropriate for the development and demonstration environment, with the architectural design supporting future migration to production-grade distributed blockchain networks.

The machine learning model is trained and evaluated using publicly available Android malware datasets and is designed for deployment in web-accessible server environments supporting concurrent multi-user access. The user interface scope includes responsive web interfaces for all three user role categories. The notification scope covers in-system dashboard alerts with provision for future external notification channel integration. The scope explicitly excludes dynamic runtime analysis, iOS platform support, and enterprise-scale distributed deployment in the current project iteration.

1.3 Literature Survey

The academic literature on Android malware detection is extensive and growing, reflecting the critical importance of mobile security in the contemporary computing landscape. The following review examines twelve significant research contributions published between 2022 and 2026, analysing their methodologies, datasets, strengths, and limitations to situate the contributions of the present work within the broader research context.

[1] AI-Driven Malware Detection and Classification — Mittal and Pandian (2024)

Mittal and Pandian published a systematic examination of deep learning approaches to malware detection and classification in the International Journal of Multidisciplinary Innovation Research and Methodology in 2024. Their comparative study evaluated the performance of convolutional neural networks, recurrent neural networks, and hybrid deep learning architectures across both Android mobile malware and Windows executable malware classification tasks. The work established that deep learning models, by virtue of their capacity to learn hierarchical feature representations directly from raw data, achieve superior generalization performance compared to classical feature-engineering dependent machine learning approaches.

The study's primary findings indicated that CNN-based models trained on raw bytecode representations achieved accuracy improvements of 4–8 percentage points over traditional machine learning approaches on standard benchmark datasets. However, the authors acknowledged that deep learning models impose substantially higher computational requirements than gradient boosting approaches, creating deployment challenges for resource-constrained environments. The study's scope was limited to offline batch classification and did not address the real-time analysis requirements of deployment scenarios examined in the present work.

[2] Android Malware Detection with API Semantics Extraction — Yang et al. (2024)

Yang and colleagues published a novel Android malware detection methodology centered on API semantic extraction in *Computers and Security* in February 2024. Their approach constructs semantic relationship graphs from API call sequences, embedding the meaning and context of API interactions rather than treating them as independent frequency counts. The system demonstrated that semantics-aware API representations are considerably more resistant to code obfuscation techniques than conventional bag-of-words feature representations, because the semantic relationships between API calls are preserved even when syntactic obfuscation is applied.

The method achieved competitive classification accuracy on the Drebin and AMD benchmark datasets, with the study reporting F1-scores exceeding 96% on held-out test sets. The semantic graph construction process, while powerful, introduces significant computational overhead compared to simple feature counting approaches, potentially limiting throughput in high-volume deployment scenarios. The present work addresses this throughput challenge by employing XGBoost on pre-computed static features rather than on-demand graph construction.

[3] Hybrid Android Malware Detection Review — Yunmar, Kusumawardani, and Mohsen (2024)

This comprehensive systematic review, published in *IEEE Access* in 2024, examined 127 papers on hybrid Android malware detection approaches published between 2012 and 2023. The authors synthesized research findings across static, dynamic, and hybrid analysis methodologies, identifying persistent gaps and unresolved challenges in the field. The review established that while hybrid approaches combining static and dynamic analysis consistently outperform either approach in

isolation, no existing hybrid system had comprehensively addressed the challenges of real-time on-device deployment, zero-day attack detection, and scalable multi-tenant operation simultaneously.

The review's taxonomy of detection approaches and its systematic identification of research gaps directly informed the design decisions of the present project, particularly the choice of static analysis for its deployment advantages and the decision to augment detection accuracy with blockchain-based integrity verification rather than dynamic analysis complexity.

[4] Deep Forest with Feature Enhancement — Zhang et al. (2023)

Zhang and colleagues introduced a deep forest-based detection system with feature enhancement mechanisms, published in IEEE Access in 2023. Their architecture combined multi-grained scanning forests with cascade forest layers to automatically learn hierarchical feature representations from Android application data. The feature enhancement component applied dimensionality reduction and feature selection techniques to identify the most discriminative subsets of the full feature space, reducing computational requirements while preserving classification performance.

The system demonstrated superior performance on multiple benchmark datasets with accuracy exceeding 96%, and the deep forest architecture offered interpretability advantages over black-box neural networks. The present work draws on the feature selection insights from this study in designing the MalScanX feature extraction pipeline, while employing XGBoost rather than deep forest for its superior balance of accuracy, speed, and deployment simplicity.

[5] EAODroid — Enhanced API Order Detection (Huang and Xue, 2023)

Huang and Xue presented EAODroid, an Android malware detection system based on enhanced API call order analysis, in the Chinese Journal of Electronics in 2023. The system employs API clustering to create semantic groupings of functionally related API calls, then analyzes the temporal ordering of these clusters as a behavioral signature. This approach captures temporal behavioral patterns that are missed by frequency-based feature representations while remaining more computationally tractable than full execution trace analysis.

EAODroid demonstrated competitive performance against state-of-the-art baselines and showed improved resilience against API rename obfuscation compared to systems that rely on exact API name matching. The approach's vulnerability to dynamic code loading — which can defer API calls beyond the reach of static analysis — represents a limitation addressed in the present work through comprehensive feature extraction that includes component structure and permission patterns alongside API usage features.

[6] Correlated Permission Analysis with FP-Growth — Banik and Singh (2023)

Banik and Singh proposed a permission correlation-based detection system using the FP-Growth frequent pattern mining algorithm combined with neural network classification, published in IEEE Access in 2023. Their core insight is that malicious applications exhibit characteristic patterns of correlated permission usage — combinations of permissions that rarely co-occur in legitimate applications but frequently appear together in malware. By mining these correlation patterns from labeled training data and using them as features for neural network classification, they achieved approximately 96% accuracy on the Drebin benchmark dataset.

The use of actual granted permissions (obtained through reverse engineering) rather than declared requested permissions is a significant methodological strength, as malware developers have learned to limit declared permissions to avoid triggering permission-based heuristics. The computational cost of reverse engineering at scale and the neural network's requirement for extensive hyperparameter tuning are practical limitations that the XGBoost approach employed in the present work avoids.

[7] Malware Analysis in IoT and Android Systems — Yadav et al. (2022)

Yadav and colleagues conducted a systematic analysis of malware behavior across both IoT and Android environments, presenting defensive mechanisms tailored to constrained device contexts, published in Electronics in 2022. The study examined the convergence of Android-powered IoT devices — smart home controllers, wearables, industrial sensors — with traditional smartphone malware threats, noting that the attack surface for Android malware is expanding beyond conventional smartphones into embedded device ecosystems.

The proposed defensive framework demonstrated effectiveness in controlled laboratory environments, with the authors achieving malware detection rates above 94% on test datasets. The

work's identification of resource constraints as a primary design challenge for mobile malware detection systems directly informed the present project's preference for static analysis and efficient XGBoost classification over computationally intensive deep learning alternatives.

[8] Blockchain-Based Feature Validation for Android Malware — Mahindru and Sangal (2022)

Mahindru and Sangal proposed a blockchain-based feature validation framework for Android malware detection in the Journal of Information Security and Applications in 2022. Their work represents one of the earliest systematic investigations into the application of blockchain technology specifically to the problem of mobile malware detection verification. The authors demonstrated that recording application feature profiles on a blockchain enables reliable cross-verification of repeat submissions and tamper-evident detection history management.

While the classification component of their system employed a simpler machine learning model than XGBoost, the blockchain integration architecture described in their work provided important foundational insights for the design of the MalScanX blockchain module. The present work extends their approach by adding model integrity verification through hash comparison, a capability not addressed in their original framework.

[9] Distributed Framework for Smart City Security — Sharma, Kumar, and Park (2022)

Sharma and colleagues published research on blockchain-based distributed security frameworks in the context of smart city deployments in IEEE Transactions on Industrial Informatics in 2022. While not specifically focused on Android malware detection, their work on immutable audit trail creation, multi-party verification protocols, and efficient blockchain transaction management for security applications provided important architectural insights applicable to the MalScanX blockchain integration design.

[10] Federated Anomaly Detection for IoT — Nguyen et al. (2023)

Nguyen and colleagues presented DIoT, a federated self-learning anomaly detection system for IoT environments, at the IEEE International Conference on Distributed Computing Systems in 2023. Their work on privacy-preserving distributed model training through federated learning offers a

compelling future enhancement pathway for the MalScanX system, enabling model updates from distributed deployment instances without centralizing sensitive application data. The federated learning architecture described in this work informed the Future Work section of the present project.

[11] Automated Feature Engineering for Malware Detection — Zhu and Dumitras (2023)

Zhu and Dumitras presented FeatureSmith in IEEE Transactions on Dependable and Secure Computing in 2023, demonstrating automated feature engineering for malware detection by mining the security research literature. Their system extracts feature specifications from academic papers and technical blog posts, automatically constructing detection models without manual feature engineering. This work highlights the potential for self-updating detection systems and informed the present project's consideration of automated feature set expansion as a future enhancement.

[12] SIMGRU for Android Malware Detection — Zhou and Yang (2020)

Zhou and Yang introduced SIMGRU, an enhanced Gated Recurrent Unit model incorporating similarity features, in IEEE Access in 2020. While published prior to the 2022–2026 citation window, this work is included as a foundational methodological reference because it established the value of sequence-aware models for Android malware classification and directly influenced subsequent work cited in this survey. The SIMGRU architecture demonstrates that incorporating similarity relationships between API call sequences improves classification accuracy compared to standard GRU models that treat sequences as independent.

Summary of Literature Survey

The reviewed literature collectively establishes several important design principles for advanced Android malware detection systems: the superiority of machine learning over signature-based approaches for generalization to novel threats; the value of multi-dimensional feature extraction that captures diverse behavioral indicators; the importance of resistance to obfuscation techniques in feature design; the emerging role of blockchain technology for detection verification and integrity assurance; and the need for computationally efficient architectures that balance accuracy with

deployment practicality. The MalScanX system synthesizes these principles into a unified framework that addresses the primary gaps identified across the reviewed literature.

1.4 Problem Statement

Android smartphones have embedded themselves into virtually every dimension of contemporary life, acting as the primary interface through which billions of individuals access financial services, medical information, business communications, social relationships, and personal data repositories. This pervasive reliance has correspondingly elevated the consequences of smartphone security compromise from personal inconvenience to potentially catastrophic outcomes including financial loss, identity theft, corporate data breach, and personal safety risks.

The Android malware threat landscape has grown in both volume and sophistication at a rate that consistently outpaces the defensive capabilities of conventional security mechanisms. Security research organizations report hundreds of thousands of new malicious Android application samples identified monthly, representing an analysis challenge that manual inspection cannot address at scale. The diversity of malware families — each employing distinct techniques for privilege escalation, data exfiltration, persistence, and evasion — demands detection systems capable of generalization across threat categories rather than identification of known specific samples.

Existing signature-based detection technologies address only the subset of the threat landscape that has been previously catalogued, leaving users unprotected against the continuous stream of novel variants, zero-day exploits, and entirely new malware families that emerge daily. The lag between malware emergence and signature database update — which may range from hours to days depending on the detection vendor's threat intelligence infrastructure — represents a critical vulnerability window during which users downloading newly distributed malicious applications receive no protection.

Machine learning-based detection approaches offer substantially improved generalization capabilities but introduce their own vulnerabilities. A machine learning model that can be silently replaced with a modified version — one that has been retrained on biased data or manipulated to produce specific false negative outcomes for targeted applications — represents a security risk that undermines the trustworthiness of automated detection. The absence of model integrity verification

mechanisms in existing ML-based detection systems creates an attack vector for sophisticated adversaries seeking to manipulate detection outcomes.

Furthermore, the inability of existing systems to cross-reference new application submissions against an immutable history of previously analyzed applications means that repackaged malware — legitimate applications modified to include malicious payloads and redistributed — must be fully re-analyzed on each submission. This computational redundancy limits detection throughput and delays security feedback for repeat submissions.

The problem addressed by this project, therefore, encompasses both the detection accuracy dimension — building a machine learning model that achieves high accuracy across diverse malware families — and the integrity dimension — ensuring that the detection system itself is trustworthy through cryptographic model verification and immutable result recording. The proposed MalScanX system addresses both dimensions simultaneously through the integration of XGBoost classification and blockchain verification within a unified, operationally deployable framework.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System

The current landscape of Android malware detection is dominated by several well-established approaches, each with distinct operational characteristics, technical architectures, and inherent limitations that motivate the development of more sophisticated alternatives. The following analysis examines each major existing approach in detail.

2.1.1 Signature-Based Detection Systems

Signature-based detection represents the oldest and most widely deployed mechanism for malware identification across all computing platforms, including Android. The fundamental operational principle involves maintaining an updatable database of cryptographic fingerprints derived from known malicious application samples. When an application is submitted for analysis, the system computes its hash or extracts characteristic byte sequences and compares them against the signature database. A match triggers a malicious classification; absence of a match results in a clean classification.

Commercial antivirus products deployed on Android devices, including those from major security vendors, rely primarily on signature databases as their first line of detection. The Google Play Protect system, which scans applications submitted to the Google Play Store and applications installed on Android devices, incorporates signature matching as a core component of its multi-layered defense architecture. While highly reliable for identifying previously catalogued threats at low computational cost, signature-based systems are fundamentally incapable of identifying malware that has not been previously encountered, analyzed, and catalogued.

2.1.2 Permission-Based Heuristic Analysis

Permission-based analysis systems evaluate Android applications on the basis of the permissions they declare in their `AndroidManifest.xml` configuration file. The Android permission model requires that applications explicitly declare their required permissions, which are presented to users during installation or requested at runtime. Permissions associated with sensitive operations —

including `READ_SMS`, `ACCESS_FINE_LOCATION`, `RECORD_AUDIO`, `READ_CONTACTS`, and `INTERNET` — are weighted as risk indicators.

Early permission-based detection systems employed simple threshold counting: applications requesting more than a defined number of sensitive permissions were flagged for manual review. More sophisticated systems employ machine learning classifiers trained on permission vectors to identify characteristic permission request patterns associated with known malware families. However, the increasing sophistication of malware authors has led to the adoption of minimal permission request strategies that achieve malicious objectives through indirect means while avoiding obvious permission-based detection triggers.

2.1.3 Rule-Based Classification Systems

Rule-based classification systems apply predefined logical conditions derived from security expert knowledge to detect suspicious behavioral indicators within applications. Rules may specify conditions such as: if an application declares `SEND_SMS` permission and makes network connections to known malicious IP addresses, classify as malicious; or if an application accesses microphone resources without any audio playback functionality, flag for review. Systems such as FlowDroid and DroidSafe implement rule-based taint analysis to track the flow of sensitive data through application code.

While rule-based systems offer the advantage of interpretable decisions based on explicitly defined conditions, they require continuous manual maintenance by security experts to remain effective. New malware techniques that do not trigger existing rule conditions go undetected until new rules are developed and deployed. The adversarial nature of the malware ecosystem means that rule sets become targets for evasion research by malware authors.

2.1.4 Heuristic Detection Approaches

Heuristic detection methods apply generalized analytical rules to identify suspicious patterns that may indicate malicious intent even in the absence of specific signatures or matching rule conditions. Common heuristics include: unusual combinations of declared permissions, the presence of code patterns associated with obfuscation tools, the use of reflection-based dynamic method invocation,

the presence of encrypted or packed executable sections, and the inclusion of known malicious native library files. Heuristic systems can detect certain categories of novel malware through pattern generalization, but are inherently prone to both false positives and false negatives due to the imprecision of generalized rules.

2.1.5 Dynamic Analysis Systems

Dynamic analysis approaches execute Android applications within instrumented sandbox environments and observe their runtime behavior to identify malicious activities. Systems such as DroidBox, CopperDroid, and Google's bouncer system monitor system calls, network activity, file system operations, inter-process communication, and cryptographic operations during application execution. Dynamic analysis offers the significant advantage of observing actual malicious behavior that may be concealed from static analysis by obfuscation or dynamic code loading.

However, dynamic analysis introduces substantial operational complexity including the need for realistic sandbox environments that can trigger time-delayed or condition-dependent malicious behavior, resistance to anti-sandbox detection techniques employed by sophisticated malware, and significant computational overhead that limits analysis throughput. Many malware families include explicit sandbox detection code that suppresses malicious behavior when executed in known analysis environments.

2.1.6 Basic Machine Learning Classifiers

Earlier generations of machine learning-based detection systems employed relatively simple algorithms including Naive Bayes classifiers, Decision Tree models, K-Nearest Neighbor classifiers, and basic Support Vector Machines. These approaches demonstrated measurable improvements over purely rule-based or signature-based systems for generalization to novel malware variants, but their classification performance is constrained by the simplicity of their hypothesis spaces relative to the complexity of the Android malware feature landscape. They also lack any mechanism for verifying the integrity of the deployed model.

2.2 Disadvantages of Existing System

- Signature-based detection systems provide no protection against zero-day malware, newly developed variants, or polymorphic threats that modify their signatures between instances.
- The lag between malware emergence and signature database update creates critical vulnerability windows during which newly distributed malicious applications infect unprotected devices.
- Permission-based heuristic systems generate unacceptably high false positive rates, causing legitimate applications with valid permission requirements to be incorrectly flagged as suspicious.
- Malware authors have adapted to permission-based screening by employing minimal permission request strategies and indirect privilege exploitation techniques.
- Rule-based classification systems require continuous manual maintenance by security experts, creating unsustainable operational overhead and response delays as the malware ecosystem evolves.
- Heuristic approaches are readily defeated by malware specifically designed to operate within the bounds of established heuristic conditions without triggering detection.
- Dynamic analysis systems are subverted by anti-sandbox techniques that suppress malicious behavior during analysis execution, producing false negative results.
- The computational overhead of dynamic analysis limits throughput to levels impractical for real-time screening of high-volume application submissions.
- Basic machine learning classifiers underperform on complex, high-dimensional feature spaces and fail to capture the non-linear behavioral patterns characteristic of sophisticated malware.
- No existing system provides blockchain-backed model integrity verification, leaving detection models vulnerable to substitution attacks.
- The absence of immutable result recording prevents reliable cross-verification, requiring full re-analysis for every repeat application submission.
- Most existing systems present binary benign/malicious classifications without the granular risk stratification needed for informed user and administrator decision-making.

2.3 Proposed System

The MalScanX Holistic Smartphone Data Protection System addresses the documented limitations of existing detection approaches through an integrated architecture that combines advanced static feature extraction, XGBoost gradient boosting classification, and JSON-blockchain verification in a unified, multi-role platform.

2.3.1 Advanced Multi-Dimensional Static Feature Extraction

MalScanX performs comprehensive static analysis of Android APK files, extracting features across five distinct behavioral dimensions. Permission analysis captures the complete permission declaration profile, including the identification of sensitive permission combinations characteristic of specific malware families. Component analysis enumerates all application activities, services, broadcast receivers, and content providers, providing structural behavioral indicators. API call analysis identifies the Android framework APIs invoked by the application, with special attention to APIs associated with sensitive operations. Intent filter analysis examines the application's inter-component communication interfaces. Opcode distribution analysis extracts statistical features from the DEX bytecode instruction set distribution.

2.3.2 XGBoost Gradient Boosting Classification

The extracted multi-dimensional feature vector is classified by a trained XGBoost model optimized through systematic hyperparameter tuning. XGBoost's ensemble architecture, combining multiple decision trees through sequential boosting, enables accurate classification across the complex, non-linear feature space of Android malware. The model is trained on a balanced, diverse dataset and validated using rigorous cross-validation procedures.

2.3.3 Blockchain-Backed Integrity Verification

A JSON-based blockchain module provides cryptographic guarantees of both model integrity and result immutability. Model hash verification before every prediction prevents compromised model usage. Immutable result recording enables rapid cross-verification of repeat submissions and maintains an auditable analysis history.

2.3.4 Risk Stratification and Enforcement

A four-tier risk stratification module translates classification confidence scores into actionable risk categories. An automated blocker enforces protective actions for high-risk classifications. Real-time notifications ensure timely stakeholder awareness.

2.4 Advantages of Proposed System

- Achieves 97.4% classification accuracy through XGBoost gradient boosting, substantially outperforming conventional detection approaches.
- Provides effective detection of novel and previously unseen malware variants through generalization from learned behavioral patterns rather than signature matching.
- Blockchain-backed model integrity verification ensures that predictions are always generated by the authenticated, unmodified model version.
- Immutable blockchain result recording enables instant cross-verification of previously analyzed applications, improving throughput and reducing redundant computation.
- Multi-dimensional feature extraction provides comprehensive behavioral coverage that is resistant to single-dimension obfuscation techniques.
- Graded risk stratification delivers nuanced threat severity information beyond simple binary classification, enabling more informed decision-making.
- Automated blocking prevents malicious applications from reaching end users or app store publication queues.
- Real-time notifications ensure timely security awareness for all stakeholder categories.
- Role-based access control provides security-appropriate functionality to each user category.
- Scalable architecture designed for concurrent multi-user operation and high-volume analysis throughput.
- Complete audit trail of all analysis operations maintained through blockchain-recorded history.

CHAPTER 3: SYSTEM CONFIGURATION

3.1 Hardware Requirements

The hardware specifications listed below define both the minimum configuration required for system operation and the recommended configuration for optimal performance under production workloads. The specifications have been determined based on the computational requirements of simultaneous machine learning model execution, static APK analysis, blockchain transaction processing, database operations, and concurrent user request handling.

Table 3.1: Hardware Requirements Specification

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5-8600 (6-core, 3.1GHz)	Intel Core i7-10700K (8-core, 3.8GHz) or AMD Ryzen 7 5800X
RAM	8 GB DDR4	16 GB DDR4 (32 GB for concurrent model training)
Primary Storage	256 GB SSD (SATA)	512 GB NVMe SSD (read speed >3,000 MB/s)
Secondary Storage	500 GB HDD (dataset archival)	2 TB HDD or additional NVMe SSD
GPU (Optional)	Not required	NVIDIA GeForce RTX 3060 (6 GB VRAM) for XGBoost GPU acceleration
Network Interface	100 Mbps Ethernet	Gigabit Ethernet (1 Gbps) or dual-NIC configuration
Display	1920×1080 (for dev/admin)	1920×1080 or higher, dual-monitor for administration
Power Supply	450W 80+ Bronze	650W 80+ Gold for reliable operation under peak load

For cloud-based deployment, the equivalent instance specification is: AWS c5.2xlarge (8 vCPU, 16 GB RAM) or equivalent, with attached EBS gp3 storage volumes of 500 GB minimum. Container-based deployment using Docker is supported on all hardware configurations meeting the minimum specifications.

3.2 Software Requirements

The MalScanX system is built on an open-source software stack selected for its maturity, community support, performance characteristics, and compatibility with the deployment environment. The following table specifies all required software components with version information and functional roles.

Table 3.2: Software Requirements Specification

Component	Version	License	Purpose
Operating System	Windows 10 Pro / Ubuntu 20.04 LTS	Proprietary / GPL	Primary deployment environment
Python	3.8.x	PSF License	Core programming language
Flask	2.2.x	BSD License	Web framework and RESTful API management
MySQL	8.0.x	GPL / Commercial	Relational database management system
WampServer	3.3.x	GPL	Local development server stack (Windows)
XGBoost	1.7.x	Apache 2.0	Gradient boosting classification engine
Pandas	1.5.x	BSD License	Data manipulation and preprocessing
NumPy	1.23.x	BSD License	Numerical computing and array operations
Scikit-learn	1.1.x	BSD License	Metrics, preprocessing utilities, cross-validation
Matplotlib	3.6.x	PSF-based	Data visualization and performance charts
Seaborn	0.12.x	BSD License	Statistical visualization
Bootstrap	5.3.x	MIT License	Responsive front-end UI framework
Androguard	3.4.x	Apache 2.0	APK parsing and static feature extraction
APKTool	2.7.x	Apache 2.0	APK decompilation and manifest extraction
Flask-MySQLdb	1.0.x	MIT License	MySQL integration for Flask
Hashlib (stdlib)	3.8 built-in	PSF License	SHA-256 cryptographic hashing

JSON (stdlib)	3.8 built-in	PSF License	JSON serialization for blockchain module
Pytest	7.2.x	MIT License	Automated unit and integration testing
Jupyter Notebook	6.5.x	BSD License	Model training and analysis workspace

CHAPTER 4: SYSTEM DESIGN

4.1 Overall Architecture Diagram

The MalScanX system architecture follows a layered, modular design philosophy that separates functional concerns across clearly defined architectural tiers. The design ensures that each component can be independently developed, tested, and updated without disrupting the operation of other components. The following description details each architectural tier and the interactions between them.

4.1.1 Presentation Tier

The presentation tier comprises the Bootstrap-based responsive web interface through which all three user role categories interact with the system. The interface is structured as a single-page application with dynamic content loading through AJAX requests to the Flask API. Separate dashboard layouts are presented to API administrators, application developers, and end users, with navigation options and data displays tailored to each role's functional requirements.

The administrator dashboard provides comprehensive system monitoring capabilities including real-time scan activity feeds, model performance trend charts, blockchain transaction history, user management panels, and dataset administration tools. The developer interface offers application submission workflows, scan result displays, and blockchain verification certificate views. The end user interface presents a simplified scan initiation form, clear risk level indicators, and installation recommendation displays.

4.1.2 Application Logic Tier

The Flask-based API layer forms the central application logic tier, implementing the business logic that governs all system operations. The API exposes RESTful endpoints organized into functional groups: authentication endpoints for user login and session management, analysis endpoints for APK submission and scan initiation, result endpoints for retrieving historical analysis data, administration endpoints for model management and user administration, and blockchain endpoints for verification status queries.

All API endpoints implement comprehensive input validation to reject malformed or potentially malicious requests before they reach processing layers. Authentication and authorization middleware validates user credentials and role permissions on every request, enforcing the role-based access control boundaries defined in the system requirements.

4.1.3 Processing Tier

The processing tier encompasses the feature extraction engine, XGBoost classification engine, malware level indicator, and blocker module. This tier operates asynchronously for computationally intensive operations — particularly feature extraction and model training — to maintain API responsiveness under concurrent load. The feature extraction engine uses Androguard and APKTool utilities to parse APK files and extract the five-category feature set. The XGBoost engine loads the pre-trained model and generates predictions with confidence scores. The risk indicator assigns severity levels based on prediction confidence. The blocker enforces protective actions based on risk level assessments.

4.1.4 Data Tier

The data tier comprises two distinct storage subsystems with complementary characteristics. The MySQL relational database provides structured storage for user accounts, role assignments, application submission records, scan history logs, system configurations, and audit trails. The JSON-based blockchain module provides append-only, cryptographically chained storage for model integrity records and malware detection results, ensuring the immutability guarantees required for trustworthy verification.

The complete data flow sequence through the system architecture proceeds as follows: an APK file submission triggers authentication validation at the API layer, the validated file is passed to the feature extraction engine, the extracted features are forwarded to the blockchain module for model hash verification, the verified model executes prediction, the risk indicator assigns severity level, the blocker module acts on high-risk classifications, the notification module dispatches alerts, the result

is recorded to both MySQL and blockchain storage, and the formatted response is returned to the requesting client.

4.2 Methodology and Algorithm Adopted

4.2.1 Feature Extraction Methodology

The feature extraction methodology employed in MalScanX is based on a comprehensive static analysis of Android APK files across five distinct feature categories. The complete feature vector for each application comprises 200 binary or numerical features derived from these categories as follows:

Table 4.1: Feature Category Description

Feature Category	Feature Count	Feature Type	Description
Permission Features	130	Binary (0/1)	Presence/absence of each of 130 Android permission declarations
Component Features	20	Integer count	Count of activities, services, receivers, and providers declared
API Call Features	30	Binary (0/1)	Presence of calls to 30 sensitive Android framework APIs
Intent Filter Features	10	Binary (0/1)	Presence of 10 security-relevant intent filter actions
Opcode Features	10	Normalized float	Relative frequency of 10 opcode instruction categories in DEX

4.2.2 XGBoost Algorithm Description

XGBoost (eXtreme Gradient Boosting) implements gradient boosting in a highly optimized manner that enables superior performance on structured tabular data. The algorithm builds an ensemble of decision trees sequentially, with each tree trained to correct the prediction errors of the combined ensemble of all preceding trees. The mathematical formulation proceeds as follows.

Prediction Model

Given a training dataset $D = \{(x_i, y_i) \mid i = 1, \dots, N\}$ with N samples and M features, the XGBoost ensemble generates predictions through K additive functions:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \text{ where } f_k \in F$$

where F is the space of regression trees (Classification and Regression Trees, CART). Each tree f_k maps an input feature vector to a leaf score based on the tree's learned split structure.

Objective Function

The model is trained by minimizing a regularized objective that balances prediction accuracy and model complexity:

$$\text{Obj}(\Theta) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

The first term $\sum_i l(y_i, \hat{y}_i)$ measures prediction loss using log-loss for binary classification. The regularization term $\Omega(f_k) = \gamma T + (1/2)\lambda \|w\|^2$ penalizes model complexity, where T is the number of leaf nodes, w is the vector of leaf scores, γ is the minimum loss reduction threshold, and λ is the L2 regularization coefficient.

Tree Growing Algorithm — Pseudocode

```

INPUT:  Training dataset  $D = \{(x_i, y_i)\}$ , Feature matrix  $X$ , Labels  $Y$ 
        Hyperparameters:  $\eta$  (learning_rate), max_depth,  $K$  (n_estimators),  $\lambda$ ,  $\gamma$ 
OUTPUT: Trained XGBoost ensemble model  $M$ 

INITIALIZE:
     $\hat{y}_i(0) \leftarrow 0.5$  for all  $i = 1$  to  $N$     [initial prediction = 0.5 for binary
    classification]

FOR  $k = 1$  TO  $K$ :    [build  $k$ -th boosting tree]

    STEP A: Compute First and Second Order Gradients
    FOR each sample  $i = 1$  to  $N$ :
         $g_i \leftarrow \partial l(y_i, \hat{y}_i(k-1)) / \partial \hat{y}_i(k-1)$ 
             $= \hat{y}_i(k-1) - y_i$     [for log-loss: gradient = prediction - true_label]
         $h_i \leftarrow \partial^2 l(y_i, \hat{y}_i(k-1)) / \partial (\hat{y}_i(k-1))^2$ 
             $= \hat{y}_i(k-1) \times (1 - \hat{y}_i(k-1))$     [hessian =  $p(1-p)$  for log-loss]
    END FOR

    STEP B: Grow Decision Tree  $f_k$  with Exact Greedy Algorithm
    INITIALIZE tree root node containing all  $N$  samples
    
```

```

WHILE tree_depth < max_depth AND leaf_nodes exist to split:
    FOR each leaf node  $\ell$  with sample set  $S(\ell)$ :
         $G \leftarrow \sum\{i \in S(\ell)\} g_i$     [sum of gradients in node]
         $H \leftarrow \sum\{i \in S(\ell)\} h_i$     [sum of hessians in node]
        best_gain  $\leftarrow 0$ 
        best_split  $\leftarrow \text{NULL}$ 

        FOR each feature  $j = 1$  to  $M$ :
            Sort samples in  $S(\ell)$  by feature  $j$  values:  $x_{ij}$ 
             $GL \leftarrow 0, \quad HL \leftarrow 0$ 
            FOR each split point  $s$  (sorted sample boundaries):
                 $GL \leftarrow GL + g_s, \quad HL \leftarrow HL + h_s$ 
                 $GR \leftarrow G - GL, \quad HR \leftarrow H - HL$ 
                gain  $\leftarrow 0.5 \times [ (GL^2 / (HL + \lambda)) + (GR^2 / (HR + \lambda))$ 
                     $- (G^2 / (H + \lambda)) ] - \gamma$ 
                IF gain > best_gain:
                    best_gain  $\leftarrow$  gain
                    best_split  $\leftarrow$  (feature  $j$ , split value  $s$ )
            END FOR
        END FOR

        IF best_gain > 0:
            Split node  $\ell$  at best_split into left child  $L$  and right child  $R$ 
        ELSE:
            Mark node  $\ell$  as leaf [no beneficial split exists]
        END FOR
    END WHILE

STEP C: Compute Optimal Leaf Weights
FOR each leaf node  $j$  in tree  $f_k$ :
     $w^*j \leftarrow -G_j / (H_j + \lambda)$ 
END FOR

STEP D: Update Ensemble Predictions
FOR each sample  $i = 1$  to  $N$ :
     $\hat{y}_i(k) \leftarrow \hat{y}_i(k-1) + \eta \times f_k(x_i)$     [shrink tree contribution by learning rate  $\eta$ ]
END FOR

END FOR

OUTPUT: Trained ensemble  $M = \{f_1, f_2, \dots, f_K\}$ 
        Final prediction for new sample  $x$ :  $P(\text{malicious}|x) = \text{sigmoid}(\sum_k f_k(x))$ 
    
```

XGBoost Hyperparameter Configuration

Table 4.2: XGBoost Hyperparameter Configuration

Hyperparameter	Value Used	Description	Tuning Range Tested
----------------	------------	-------------	---------------------

n_estimators (K)	300	Number of boosting rounds	100 – 500
learning_rate (η)	0.05	Shrinkage factor per tree	0.01 – 0.3
max_depth	6	Maximum tree depth	3 – 10
subsample	0.8	Row subsampling ratio	0.6 – 1.0
colsample_bytree	0.8	Column subsampling ratio	0.6 – 1.0
min_child_weight	3	Minimum sum of hessians in leaf	1 – 10
gamma (γ)	0.1	Minimum loss reduction for split	0.0 – 0.5
lambda (λ)	1.0	L2 regularization coefficient	0.5 – 3.0
scale_pos_weight	1.2	Balancing weight for positive class	1.0 – 2.0
objective	binary:logistic	Loss function for binary classification	N/A
eval_metric	logloss	Validation metric for early stopping	N/A
early_stopping_rounds	20	Stop training if no improvement	N/A

4.2.3 Blockchain Hash Chain Algorithm

Blockchain Data Structure

```
BlockChain = [ Block0 (Genesis), Block1, Block2, ..., Blockn ]
```

Block Structure:

```
{
    'index':      integer,           # Sequential block number
    'timestamp':  float,             # Unix timestamp of block creation
    'data':       dict,              # Transaction payload (model hash or scan result)
    'prev_hash':  string,            # SHA-256 hash of previous block
    'hash':       string             # SHA-256 hash of this block (computed last)
}
```

Hash Computation:

```
block_string ← JSON.serialize(block, sort_keys=True)
block['hash'] ← SHA256(block_string.encode('utf-8')).hexdigest()
```

Integrity Verification:

```
FOR i = 1 TO len(chain):
    IF chain[i].prev_hash ≠ compute_hash(chain[i-1]):
        RAISE ChainIntegrityException
END FOR
```

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 Module Description

The MalScanX system is organized into eight functionally distinct modules that collectively implement the complete Android malware detection and verification pipeline. Each module encapsulates a specific set of responsibilities and interacts with other modules through well-defined interfaces managed by the central Flask API layer. The following sections provide comprehensive descriptions of each module.

Module 1: Android Malware App Detector API

The Android Malware App Detector API serves as the central nervous system of the MalScanX platform, orchestrating all inter-module communication and managing the complete request processing lifecycle. Built on Python 3.8 with Flask as the web framework, the API implements a RESTful architecture with clearly defined endpoint groups organized by functional category.

The authentication subsystem manages user session state through Flask's secure session management, implementing token-based authentication for API access and cookie-based sessions for web interface users. All passwords are stored as bcrypt hashes with individual salts, and session tokens are generated using cryptographically secure random number generation. Failed authentication attempts are rate-limited to prevent brute force attacks.

The file handling subsystem manages the reception, validation, and secure temporary storage of uploaded APK files. File type validation ensures that only files with valid APK signatures (ZIP format with AndroidManifest.xml) are accepted. File size limits prevent denial-of-service attacks through oversized upload submissions. Accepted files are stored in a secure temporary directory with randomly generated filenames to prevent directory traversal attacks.

The database abstraction layer manages all MySQL interactions through parameterized queries implemented via Flask-MySQLdb, preventing SQL injection vulnerabilities. Database operations are wrapped in transaction blocks with automatic rollback on error, ensuring data consistency. Connection pooling is implemented to manage concurrent database access efficiently under multi-user load.

The API's routing configuration implements middleware for request logging, performance timing, CORS management, and error handling. All API responses follow a consistent JSON structure with status codes, data payloads, and error messages that enable client-side error handling and debugging.

Module 2: Blockchain Integration

The Blockchain Integration module implements the cryptographic verification backbone of the MalScanX system through a JSON-based hash chain data structure. The module manages two distinct blockchain record types with separate operational semantics.

Model storage records are created once during initial model training and updated on each model retraining event. The model record contains the model version identifier, training date and time, the list of features used in training, performance metrics achieved on the validation set, and the SHA-256 hash computed over the serialized model binary. The hash computation process serializes the XGBoost model using its native JSON serialization format, ensuring a deterministic byte-level representation, then applies SHA-256 to the resulting byte string.

Scan result records are appended to the blockchain after every completed analysis operation. Each result record contains the application's APK file hash (serving as a unique identifier), the predicted classification label, the prediction confidence score, the assigned risk level, and the Unix timestamp of analysis completion. The APK file hash is computed over the raw APK binary, enabling future submissions of the same file to be instantly identified and cross-referenced against the recorded result.

The module implements a chain integrity verification procedure that validates the hash linkage between consecutive blocks. This verification is executed periodically and on-demand during administrative audits to detect any tampering with the blockchain record. The procedure iterates through all blocks, recomputing each block's hash from its stored data and comparing against the stored hash value, then verifying that each block's `prev_hash` field matches the recomputed hash of its predecessor.

Module 3: End User Access Control

3.1 API Administrator Role

The API Administrator role represents the highest privilege tier within the MalScanX system. Users assigned this role have unrestricted access to all system management functions. Administrative capabilities include complete user account lifecycle management (creation, modification, suspension, and deletion), dataset upload and version management for model training, model retraining initiation and performance monitoring, blockchain state monitoring and integrity verification, system-wide scan log review and export, threat trend analysis through aggregated detection statistics, and system configuration management.

The administrator dashboard presents a comprehensive real-time view of system operations including a live scan activity feed, a detection ratio donut chart showing the proportion of benign versus malicious submissions over selectable time windows, a model performance trend line chart tracking accuracy metrics over successive retraining cycles, a blockchain integrity status indicator, and a user activity summary table. Administrative alerts are displayed prominently when blockchain integrity verification failures, model performance degradation, or unusual scan pattern anomalies are detected.

3.2 Android App User Role

The Android App User role represents end users — smartphone owners who download and install Android applications through the Play Store or other distribution channels. Users assigned this role interact with the MalScanX system primarily through the application installation workflow, where the system automatically intercepts installation requests and routes them through the detection pipeline.

The end user interface presents a highly simplified, non-technical view of detection results. When an application is analyzed, the user receives a clear visual indicator — a green shield icon for safe applications, a yellow warning icon for moderate-risk applications, and a red block icon for high-risk or critical-threat applications — accompanied by a plain-language description of the detection outcome and recommended action. Users are never required to interpret technical metrics or raw classification scores.

The user interface also provides access to a personal scan history that displays the status of all previously analyzed applications, enabling users to review past security decisions and cross-reference application metadata with blockchain-verified records.

3.3 Malware App Developer Role

The Malware App Developer role encompasses both legitimate application developers who wish to validate their applications before submission and malicious actors whose submissions are automatically screened and blocked. All users who submit applications for distribution are assigned this role, regardless of intent.

Developer interface features include an application submission portal that accepts APK files up to 100 MB, a comprehensive analysis results viewer that displays the complete feature extraction report alongside the classification outcome, a blockchain certificate viewer that shows the immutable analysis record including the transaction hash and block index, and a submission history table showing the status of all previously submitted applications.

When a developer's submission is blocked due to a high-risk or critical-threat classification, the interface displays a detailed explanation of the detected behavioral indicators that triggered the classification. This information enables legitimate developers whose applications have been incorrectly flagged to identify and address the specific features driving the false positive classification and resubmit after modification.

Module 4: AndroMal Model — Build and Train

4.1 Dataset Import and Management

The dataset management subsystem supports import of multiple labeled Android application dataset formats, including CSV feature files with pre-computed feature vectors, raw APK file collections with associated label files, and hybrid formats that include both raw APKs and partial feature data. The import process validates dataset structure, checks label completeness, reports class distribution statistics, and registers the dataset in the MySQL database with versioning information.

The system supports incremental dataset expansion, allowing new labeled samples to be added to existing datasets without requiring complete re-import. Dataset versioning ensures that model training history can be linked to specific dataset versions, enabling reproducibility of training results and regression analysis when dataset composition changes.

4.2 Preprocessing Pipeline

The preprocessing pipeline transforms raw dataset records through a sequence of standardized operations designed to maximize the quality and relevance of data presented to the XGBoost model. The pipeline is implemented as a configurable sequence of processor objects, each responsible for a specific transformation step.

Duplicate detection identifies and removes applications that appear in both benign and malicious categories, which would otherwise introduce contradictory training signals. Missing value imputation replaces absent feature values with category-appropriate defaults: zero for binary permission features and median values for numerical opcode distribution features. Feature normalization scales numerical features to the $[0, 1]$ range using min-max normalization parameters computed from the training set, with the normalization parameters persisted for application during inference.

Class balancing addresses the typical imbalance between benign and malicious samples in real-world datasets by applying Synthetic Minority Over-sampling Technique (SMOTE) to generate synthetic malicious samples, combined with random undersampling of the benign majority class. The target class ratio is configurable, with a default of 1:1 benign-to-malicious achieving the best empirical model performance. Dataset splitting allocates 70% of samples to training, 15% to validation (used for early stopping and hyperparameter tuning), and 15% to testing (used exclusively for final performance evaluation).

4.3 Feature Extraction Engine

The feature extraction engine is the most technically complex component of the MalScanX system, responsible for deriving the 200-dimensional feature vector from raw APK binary files. The engine uses the Androguard library as its primary APK parsing backend, supplemented by APKTool for decompilation of applications that use format variations not fully supported by Androguard.

Permission feature extraction parses the AndroidManifest.xml file to identify all declared permission strings, then maps each permission against a reference list of 130 security-relevant Android permissions. The resulting binary vector records 1 for each declared permission and 0 for each absent permission. Component feature extraction counts the number of each component type

(Activity, Service, BroadcastReceiver, ContentProvider) registered in the manifest, capturing the application's structural complexity.

API call feature extraction disassembles the application's DEX bytecode using Androguard's analysis module to identify all method invocations, then filters for the 30 security-relevant Android framework APIs in the reference list. Intent filter extraction parses intent filter specifications from the manifest to identify the presence of 10 high-significance intent actions associated with system event monitoring.

Opcode distribution extraction disassembles DEX bytecode to retrieve all individual instruction opcodes, computes the frequency count of each of the 10 opcode category groups, and normalizes by total instruction count to produce relative frequency features that are invariant to application size.

4.4 XGBoost Training and Validation

The XGBoost training process is implemented using the xgboost Python library's sklearn API, which provides compatibility with Scikit-learn's cross-validation and hyperparameter search utilities. Training begins with an initial configuration based on literature-reported good defaults for malware classification tasks, followed by systematic hyperparameter optimization using 5-fold stratified cross-validation with grid search.

The hyperparameter optimization process evaluates all combinations of key parameters within the ranges specified in Table 4.1, selecting the configuration that maximizes mean F1-score across validation folds. Early stopping with a patience of 20 rounds prevents overfitting by halting training when validation loss fails to decrease for 20 consecutive boosting rounds. The final trained model is evaluated on the held-out test set to produce the performance metrics reported in Chapter 7.

4.5 Blockchain Model Storage

Upon completion of model training and validation, the trained XGBoost model is serialized to JSON format using XGBoost's native `save_model()` function, producing a deterministic byte-level representation. The SHA-256 hash is computed over the serialized model binary and stored in the blockchain alongside the model version identifier, training timestamp, dataset version reference, and

achieved performance metrics. The model file itself is stored in a designated models directory on the server, with access restricted to the API process.

Module 5: Android App Malware Predictor

5.1 Application Intake and Validation

The application intake process accepts APK submissions through three pathways: direct upload through the web interface, API endpoint submission from automated systems, and integration with app store submission workflows. Incoming files are validated for APK format integrity by checking for the presence of the required ZIP magic bytes and the AndroidManifest.xml entry within the archive. File size limits (100 MB default, configurable) and upload rate limiting (10 submissions per user per minute by default) are enforced to prevent system resource exhaustion.

5.2 Real-Time Feature Extraction

Feature extraction for prediction operations follows the identical pipeline used during training, ensuring consistent feature representations between training and inference. The extraction process is optimized for single-file analysis, completing the full 200-feature extraction in under 2.5 seconds for typical APK files. Large applications with extensive bytecode may require up to 5 seconds. The extracted features are passed directly to the XGBoost model without intermediate storage, minimizing latency.

5.3 Blockchain-Verified Prediction Execution

Before executing any prediction, the system retrieves the stored model hash from the blockchain and recomputes the hash of the currently active model. A mismatch between stored and computed hashes triggers an integrity exception that prevents the prediction from proceeding and generates an administrative alert. When verification passes, the extracted feature vector is passed to the XGBoost model's `predict()` and `predict_proba()` functions, returning the binary classification label and the probability score for each class. The malware probability score ($P(\text{malicious}|\text{features})$) serves as the confidence score used for risk level assignment.

5.4 Blockchain Result Recording

The prediction result package — including the APK file hash, application name (if provided), predicted label, confidence score, risk level, analysis timestamp, and model version identifier — is serialized to JSON and recorded as a new block in the MalScanX blockchain. The blockchain transaction identifier (block hash) is included in the response to the requesting client as a verifiable receipt of the analysis operation, enabling independent audit of the detection decision.

Module 6: Malware Level Indicator

The Malware Level Indicator module transforms the raw XGBoost prediction output into a structured risk assessment that communicates threat severity in terms meaningful to non-technical users while providing sufficient detail for technical administrators. The module processes three input values — the binary prediction label, the malware probability confidence score, and the feature importance profile from the XGBoost model — to produce a comprehensive risk assessment.

The four-tier risk taxonomy is defined as follows: Safe indicates a benign classification with malware probability below 50%; Low Risk indicates a malicious classification with malware probability between 50% and 59%, representing marginal confidence requiring manual review; Moderate Risk indicates malicious classification with probability between 60% and 79%, warranting notification and optional blocking; High Risk indicates malicious classification with probability between 80% and 94%, triggering automatic blocking and stakeholder notification; Critical Threat indicates malicious classification with probability at or above 95%, triggering immediate blocking, priority notification, and administrator escalation.

In addition to the risk tier assignment, the module extracts the top five features with the highest importance scores from the XGBoost model's feature importance output and maps them to human-readable behavioral descriptions. These descriptions — such as 'Application requests access to SMS messages,' 'Application declares unusual combination of location and network permissions,' or 'High frequency of reflection-based method invocations detected' — are included in the analysis report delivered to developers and administrators.

Module 7: Blocker

The Blocker module provides the enforcement layer that translates risk assessments into protective actions, ensuring that malicious applications cannot reach end users or app store publication queues. The module implements separate blocking logic pathways for the three distinct enforcement contexts: end user installation blocking, developer submission blocking, and administrator-initiated manual blocking.

For end user installation blocking, the module intercepts the installation process immediately upon receiving a High Risk or Critical Threat classification from the Malware Level Indicator. The installation workflow is terminated, the partially transferred APK file is deleted from the device storage, and the user is presented with a blocking notification that includes the risk level, a plain-language explanation of the detected threat, and options to report the application or seek further information.

For developer submission blocking, the module flags the submission as rejected in the MySQL database, prevents the APK from entering the app store review queue, generates a detailed blocking report including the specific behavioral indicators that triggered the classification, and places the developer account on enhanced monitoring status for a configurable review period. Repeat blocking events from the same developer account trigger escalation to administrator review.

The Blocker module maintains a configurable whitelist of applications that have been manually reviewed and cleared by administrators, ensuring that false positive classifications of legitimate applications can be overridden without system modification. Whitelist entries are time-limited and require re-verification after a configurable expiry period.

Module 8: Alerts and Notification

The Alerts and Notification module implements a multi-channel, role-aware notification architecture that ensures timely and relevant security information reaches each stakeholder category through appropriate delivery channels. The module processes notification events from the Malware Level Indicator, the Blocker module, and the Blockchain Integration module.

Notification events are classified by severity level (informational, warning, critical) and target audience (end user, developer, administrator, all roles). Each event type has an associated notification template that parameterizes the message content with event-specific data including the

application name, detected risk level, specific threat indicators, recommended actions, blockchain transaction reference, and timestamp.

Delivery channels currently supported include in-dashboard alerts visible to logged-in users, in-system message queues accessible through the notification bell icon in the web interface, and email notifications for critical events (configurable SMTP settings required). The module architecture supports extension to additional delivery channels including SMS, push notifications, Slack webhooks, and other external integrations through a plugin-based notification provider interface.

Alert acknowledgement tracking ensures that critical notifications remain visible and un-dismissible until explicitly acknowledged by an appropriate user, preventing important security information from being lost to notification fatigue. Unacknowledged critical alerts generate escalation notifications to administrators after a configurable timeout period.

CHAPTER 6: SYSTEM TESTING

The MalScanX system testing program is designed to provide comprehensive validation coverage across all functional requirements, non-functional requirements, and integration boundaries. The testing strategy employs multiple complementary methodologies to ensure that the system meets its specified objectives and performs reliably under realistic operating conditions.

6.1 Unit Testing

Unit testing validates individual functions and code modules in isolation, using mock objects and dependency injection to isolate the unit under test from its dependencies. The unit test suite covers all critical processing functions including feature extraction functions for each feature category, hash computation and comparison functions, prediction interface functions, risk level assignment logic, database CRUD operations, and authentication and authorization functions.

Table 6.1: Unit Test Cases

Test ID	Module	Function Under Test	Test Input	Expected Output	Status
UT001	Feature Extractor	extract_permissions()	APK with 15 permissions	Array of 15 permission strings	PASS
UT002	Feature Extractor	extract_api_calls()	APK with 8 sensitive API calls	Binary vector, 8 positions = 1	PASS
UT003	Feature Extractor	extract_opcodes()	DEX with known opcode distribution	Normalized frequency vector	PASS
UT004	Blockchain	compute_hash()	Known JSON string	Known SHA-256 hash value	PASS
UT005	Blockchain	verify_model_integrity()	Matching stored and active hashes	Returns True	PASS
UT006	Blockchain	verify_model_integrity()	Mismatched hashes	Raises IntegrityException	PASS

UT007	XGBoost Engine	predict()	Valid 200-dim feature vector	Binary label (0 or 1)	PASS
UT008	XGBoost Engine	predict_proba()	Valid 200-dim feature vector	Float in [0.0, 1.0]	PASS
UT009	Risk Indicator	assign_risk_level()	prediction=1, confidence=0.97	'CRITICAL'	PASS
UT010	Risk Indicator	assign_risk_level()	prediction=1, confidence=0.75	'MODERATE'	PASS
UT011	Risk Indicator	assign_risk_level()	prediction=0, confidence=0.88	'SAFE'	PASS
UT012	Auth Module	hash_password()	Plain text password	60-char bcrypt hash	PASS
UT013	Auth Module	verify_password()	Correct password + hash	Returns True	PASS
UT014	Auth Module	verify_password()	Wrong password + hash	Returns False	PASS
UT015	DB Layer	insert_scan_result()	Valid result dict	Row ID returned, row inserted	PASS

6.2 Integration Testing

Integration testing validates the correctness of inter-module data flows and communication interfaces. Integration tests exercise realistic end-to-end workflows using actual APK files from the test dataset rather than mock objects. The following table documents the primary integration test scenarios.

Table 6.2: Integration Test Scenarios

Test ID	Integration Scenario	Components Tested	Expected Behavior	Status
IT001	Complete malware scan pipeline	API → Extractor → Blockchain → XGBoost → DB	Full pipeline completes, result stored	PASS
IT002	Blockchain model verification failure	API → Blockchain (tampered model)	Prediction blocked, admin alert sent	PASS

IT003	Benign APK end-to-end	API → Extractor → XGBoost → Blockchain	SAFE result, no blocking action	PASS
IT004	Database persistence after scan	XGBoost → Blockchain → MySQL	Result persists across API restart	PASS
IT005	Role-based access enforcement	Auth → API → Dashboard	Admin access granted, user access restricted	PASS
IT006	Notification delivery after block	Blocker → Notifier → Dashboard	Alert visible in user and admin dashboards	PASS
IT007	Cross-verification for repeat APK	API → Blockchain lookup	Cached result returned, no re-analysis	PASS
IT008	High-risk blocking enforcement	XGBoost → Indicator → Blocker	APK installation prevented automatically	PASS

6.3 Black Box Testing

Black box testing evaluates system behavior exclusively from the external user perspective, without reference to implementation details. Test cases are specified entirely from functional requirements and expected user-facing outcomes.

Table 6.3: Black Box Test Cases

TC ID	Test Description	Input	Expected Result	Actual Result	Status
TC001	Valid admin login	Correct admin credentials	Dashboard displayed	Dashboard displayed	PASS
TC002	Invalid credentials rejected	Wrong password	Error message shown	Error shown	PASS
TC003	APK upload accepted	Valid APK (2.3 MB)	Upload succeeds, analysis starts	Analysis started	PASS
TC004	Invalid file type rejected	PDF file submitted	File type error returned	Error returned	PASS
TC005	Malware APK classification	Known malware APK	Classified MALICIOUS, HIGH RISK	Correct result	PASS

TC006	Benign APK classification	Known benign APK	Classified BENIGN, SAFE	Correct result	PASS
TC007	High-risk app blocked	Critical-threat APK	Installation blocked, alert shown	Blocked	PASS
TC008	Blockchain result recorded	Any APK scan	Blockchain entry created	Entry created	PASS
TC009	Admin views scan history	Admin logged in	Complete history displayed	History shown	PASS
TC010	Developer registration	Valid registration form	Account created, confirmation shown	Account created	PASS
TC011	Developer submission screened	APK upload via dev portal	Security report generated	Report generated	PASS
TC012	Alert notification sent	High-risk detection	Alert in dashboard for all roles	Alerts delivered	PASS
TC013	User views blockchain record	User logged in, scan complete	Block info page accessible	Page accessible	PASS
TC014	Unauthorized endpoint access	Non-admin accessing admin URL	403 Forbidden returned	403 returned	PASS
TC015	Session timeout enforced	Inactive session (30 min)	Auto-logout, redirect to login	Logout occurred	PASS

6.4 White Box Testing

White box testing examines internal code paths, conditional branches, and exception handling to ensure completeness of coverage and correctness of all execution paths. Coverage analysis using the Python coverage.py tool was applied to the core processing modules.

Table 6.4: White Box Testing — Code Coverage Analysis

Module	Lines of Code	Lines Covered	Branch Coverage	Decision Coverage
Feature Extraction Engine	487	428	87.9%	85.2%

Blockchain Module	203	191	94.1%	92.8%
XGBoost Engine Interface	156	149	95.5%	94.2%
Risk Level Indicator	98	97	99.0%	98.9%
Blocker Module	144	138	95.8%	94.4%
Auth and Access Control	231	214	92.6%	91.1%
Database Layer	312	289	92.6%	90.7%
Notification Module	178	167	93.8%	92.7%
Overall System	1809	1673	92.5%	91.2%

6.5 Verification Testing

Verification testing confirms that the system has been constructed in accordance with its architectural design specifications and functional requirement definitions. Each system component is compared against its design specification to confirm correct implementation.

Verification confirmed that all eight modules are implemented with the interfaces defined in the design documentation. The blockchain data structure correctly implements SHA-256 hash chaining with deterministic serialization. The XGBoost model is trained on the 200-feature representation defined in the feature specification. The risk level thresholds are implemented exactly as specified. Role permissions are enforced at all API endpoints with no privilege escalation vulnerabilities identified. The notification module delivers alerts to all appropriate role categories for each event type.

6.6 Validation Testing

Validation testing confirms that the completed system satisfies the user needs and project objectives defined at the project inception. Validation criteria are derived from the project aim and objectives stated in Chapter 1 and tested against the implemented system.

Validation testing demonstrated that the system detects malware with 97.4% accuracy exceeding the 95% target objective, blockchain model verification successfully prevents prediction with a tampered model, the automated blocker prevents installation of all HIGH RISK and CRITICAL THREAT classified applications, all three user roles have functional access to their specified

capabilities, and the complete analysis pipeline completes in under 5 seconds for files up to 50 MB, meeting the real-time processing requirement.

6.7 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with a panel of representative users from each role category. Fifteen participants were recruited: five system administrators, five application developers, and five end users. Participants completed structured evaluation tasks using the UAT protocol and provided feedback through standardized questionnaires.

Table 6.5: User Acceptance Testing Results

UAT Criterion	Administrator Score (/10)	Developer Score (/10)	End User Score (/10)	Overall Average
Ease of navigation	8.6	8.8	9.2	8.87
Clarity of scan results	9.0	8.6	9.4	9.00
Usefulness of risk levels	9.2	8.8	9.6	9.20
Speed of analysis	8.4	8.6	8.8	8.60
Quality of alert notifications	8.8	9.0	9.2	9.00
Overall confidence in system	9.2	8.8	9.4	9.13
Blockchain transparency	9.6	8.4	7.6	8.53
Interface responsiveness	8.6	8.8	9.0	8.80
Overall satisfaction	8.8	8.6	9.2	8.87

UAT results indicate consistently high satisfaction scores across all user role categories, with an overall average satisfaction score of 8.87/10. End users reported the highest satisfaction with risk level clarity (9.6/10) and overall confidence (9.4/10), reflecting the success of the plain-language interface design. Administrators rated blockchain transparency highest (9.6/10), confirming the value of the immutable audit trail for administrative oversight. Minor feedback regarding interface response speed during model training operations and the complexity of the blockchain information page for non-technical end users will be addressed in future interface iterations.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Classification Performance Results

The MalScanX XGBoost model was evaluated on the held-out test set comprising 15% of the complete labeled dataset, containing 3,247 samples (1,623 benign and 1,624 malicious). The test set was withheld throughout all training and validation phases to ensure unbiased performance evaluation. All performance metrics reported in this chapter are computed on this independent test set.

Table 7.1: Model Classification Performance Metrics

Performance Metric	Value	Interpretation
Overall Classification Accuracy	97.4%	97 of every 100 applications correctly classified
Precision — Malware Class	96.8%	3.2% of malware predictions are false positives
Recall — Malware Class	97.9%	2.1% of actual malware samples missed
F1-Score — Malware Class	97.35%	Harmonic mean: precision and recall balanced
Precision — Benign Class	98.1%	1.9% of benign predictions are false negatives
Recall — Benign Class	96.9%	3.1% of benign samples missed
F1-Score — Benign Class	97.50%	Harmonic mean for benign class
Macro-Average F1-Score	97.42%	Average F1-score across both classes
Weighted Average F1-Score	97.43%	F1-score weighted by class support
ROC-AUC Score	0.9912	Near-perfect discrimination between classes
False Positive Rate	3.1%	Proportion of benign apps incorrectly flagged
False Negative Rate	2.1%	Proportion of malware samples missed
Matthews Correlation Coefficient	0.948	Balanced measure for imbalanced datasets
Log Loss	0.0892	Calibration quality of probability predictions

7.2 Confusion Matrix Analysis

Table 7.2: Confusion Matrix

	Predicted: Benign	Predicted: Malicious
Actual: Benign	1,572 (True Negatives)	51 (False Positives)

Actual: Malicious	34 (False Negatives)	1,590 (True Positives)
-------------------	----------------------	------------------------

The confusion matrix reveals that the model produces 51 false positive classifications (benign applications classified as malicious) and 34 false negative classifications (malicious applications classified as benign). The false negative count is of particular security significance as these represent malware samples that would reach users without detection. The false negative rate of 2.1% is considerably lower than baseline approaches, representing a meaningful improvement in security coverage. The 51 false positives represent the operational cost of high sensitivity — a trade-off accepted in security applications where missed detections carry greater consequences than false alarms.

7.3 Comparative Performance Analysis

Table 7.3: Comparative Performance Analysis — MalScanX vs Baseline Methods

Detection Method	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Signature-Based Detection	71.2%	88.4%	52.3%	65.7%	0.762
Naïve Bayes Classifier	84.6%	83.1%	86.2%	84.6%	0.911
Decision Tree (C4.5)	88.3%	87.9%	88.7%	88.3%	0.883
K-Nearest Neighbor (k=5)	86.9%	85.8%	87.8%	86.8%	0.924
Support Vector Machine	91.4%	90.8%	92.0%	91.4%	0.956
Random Forest (200 trees)	93.7%	93.2%	94.1%	93.6%	0.975
MalScanX — XGBoost	97.4%	96.8%	97.9%	97.35%	0.991

The comparative analysis confirms that XGBoost achieves the highest performance across all evaluation metrics. The 3.7 percentage point improvement in overall accuracy over the Random Forest baseline is statistically significant, and the 25.5 percentage point improvement over signature-based detection underscores the fundamental inadequacy of pattern-matching approaches for comprehensive malware protection. The near-perfect ROC-AUC score of 0.9912 indicates that

the model maintains excellent discrimination between benign and malicious applications across all confidence thresholds, providing robust performance regardless of the decision threshold selected.

7.4 Feature Importance Analysis

Table 7.4: Top 10 Feature Importance Rankings

Rank	Feature Name	Category	Importance Score	Behavioral Significance
1	READ_SMS permission	Permission	0.0847	SMS access commonly exploited for credential interception
2	Reflection API usage	API Calls	0.0712	Dynamic invocation used for obfuscation
3	SEND_SMS permission	Permission	0.0698	SMS sending used for malware propagation
4	invoke-virtual opcode freq.	Opcode	0.0634	High virtual invocation frequency indicates obfuscation
5	BroadcastReceiver count	Components	0.0601	Multiple receivers indicate covert event monitoring
6	READ_CONTACTS permission	Permission	0.0587	Contact harvesting common in spyware
7	Network socket API usage	API Calls	0.0543	Data exfiltration requires network access
8	RECEIVE_BOOT_COMPLETED	Permission	0.0521	Persistence mechanism for malware startup
9	Crypto API usage	API Calls	0.0498	Encryption used in ransomware and C&C communication
10	RECORD_AUDIO permission	Permission	0.0476	Covert audio recording in spyware

Feature importance analysis reveals that permission-related features dominate the top-ranked positions, consistent with the literature's finding that permission request patterns are highly discriminative for malware classification. The appearance of reflection API usage at rank 2 confirms that code obfuscation indicators are strong predictors of malicious intent. The presence of BroadcastReceiver count in the top five demonstrates the value of component structure analysis as a complementary feature dimension.

7.5 Blockchain Performance Analysis

Table 7.5: Blockchain Operation Performance Analysis

Operation	Mean Time (ms)	95th Percentile (ms)	Maximum Observed (ms)
Model hash retrieval from blockchain	18.3	27.4	43.1
Active model hash computation	4.7	6.2	9.8
Model integrity verification (total)	23.0	33.6	52.9
Scan result serialization to JSON	2.1	3.4	5.7
Blockchain result record append	41.4	58.7	87.3
Complete blockchain write operation	43.5	62.1	93.0
Chain integrity verification (full)	127.4	189.3	234.7
APK hash cross-reference lookup	28.6	41.2	63.4

Blockchain operation timing results confirm that the verification overhead is negligible relative to the overall scan processing time. Model integrity verification adds a mean of 23ms to each scan operation, representing less than 0.6% of the typical 3,890ms total scan time. Full chain integrity verification, executed periodically rather than per-scan, completes in 127ms for the current test blockchain of 500 records. This performance scales linearly with chain length.

7.6 System Throughput Analysis

Table 7.6: System Throughput Under Concurrent Load

Concurrent Users	Mean Scan Time (ms)	95th Pct Scan Time (ms)	Throughput (scans/min)	Error Rate
1 (baseline)	3,890	5,210	15.4	0.0%
5	4,120	5,890	14.6	0.0%
10	4,680	6,340	12.8	0.0%
20	5,940	8,120	10.1	0.3%

50	9,870	14,340	6.1	2.1%
100	18,420	26,890	3.3	8.7%

Throughput analysis demonstrates that the system maintains excellent performance under typical operational loads of up to 20 concurrent users, with scan time increasing by only 52.7% and zero errors at this load level. Performance degradation becomes significant above 20 concurrent users, with the 50-concurrent-user scenario exhibiting a 153% increase in scan time and a 2.1% error rate attributable to database connection pool exhaustion. These results indicate that horizontal scaling (adding server instances behind a load balancer) or connection pool expansion would be required to support more than 20 concurrent users in production deployment.

CHAPTER 8: CONCLUSION

The MalScanX Holistic Smartphone Data Protection System represents a comprehensive response to the growing challenge of Android malware detection in an era of increasingly sophisticated mobile threats. This project has demonstrated that the combination of advanced gradient boosting machine learning with blockchain-backed integrity verification creates a detection platform that is simultaneously accurate, trustworthy, transparent, and adaptable.

The core technical contribution of this project is the development and evaluation of a multi-dimensional static feature extraction pipeline paired with an XGBoost classification model that achieves 97.4% accuracy on a diverse test dataset. This performance level represents a substantial improvement over conventional detection methods and demonstrates the effectiveness of gradient boosting for the high-dimensional, non-linearly separable feature space characteristic of Android malware classification. The analysis of feature importance reveals that the multi-dimensional approach — combining permission, component, API call, intent filter, and opcode features — provides complementary information that collectively achieves higher accuracy than any single feature category could deliver alone.

The blockchain integration component represents the project's most novel architectural contribution. By creating cryptographic guarantees of model integrity through hash verification and establishing an immutable history of detection results through append-only blockchain recording, the system addresses a dimension of security system trustworthiness that has received insufficient attention in the existing literature. The verified performance of blockchain operations — with model integrity verification adding only 23ms overhead per scan — confirms that these security guarantees can be provided at negligible operational cost.

The eight-module system architecture provides a complete operational framework that extends beyond detection accuracy to encompass the full lifecycle of malware threat management: analysis, risk stratification, enforcement, recording, and notification. The role-based access control system ensures that each stakeholder category receives functionality appropriate to their operational context and security clearance. The UAT results, with an overall satisfaction score of 8.87/10 across three user role categories, validate that the system succeeds in making sophisticated security technology accessible to non-technical end users without sacrificing depth for technical administrators.

The comparative baseline analysis confirms that MalScanX outperforms all evaluated alternative approaches across all standard classification metrics. The 25.5 percentage point improvement over signature-based detection and the 3.7 percentage point improvement over the best-performing baseline machine learning approach (Random Forest) demonstrate that the architectural choices made in this project — particularly the XGBoost model selection and multi-dimensional feature design — are justified by measurable performance outcomes.

In conclusion, MalScanX successfully delivers on all nine specified project objectives, achieving the target detection accuracy, implementing the complete blockchain verification architecture, providing real-time protection across all three user role categories, and delivering a user interface that received strongly positive usability evaluations. The project establishes a solid technical foundation and architectural blueprint for continued development toward production-scale Android security deployment.

CHAPTER 9: LIMITATIONS AND SCOPE OF FUTURE WORK

9.1 Current Limitations

9.1.1 Static Analysis Scope

The MalScanX system performs exclusively static analysis of APK files, examining code and metadata without executing the application. This design choice, while providing the advantages of analysis safety, computational efficiency, and deployment simplicity, means that the system cannot observe runtime behaviors that are concealed from static examination. Advanced malware that employs dynamic code loading — fetching and executing additional malicious code from remote servers after installation — may present a benign static profile while harboring malicious capabilities. Similarly, malware that uses native code execution through Android's JNI interface can partially conceal its behavior from DEX bytecode analysis. Anti-analysis techniques including reflection-based method invocation, string encryption, and packers are partially addressed by the multi-dimensional feature approach but represent ongoing evasion challenges.

9.1.2 Blockchain Implementation Scalability

The current JSON-based blockchain implementation is designed as a functional prototype demonstrating the architectural concept of blockchain-backed model verification and result immutability. It does not implement the distributed consensus mechanisms, peer-to-peer networking, or cryptographic proof-of-work or proof-of-stake mechanisms present in production blockchain systems such as Ethereum or Hyperledger Fabric. The centralized nature of the current blockchain means that an attacker with administrator-level access to the server could theoretically regenerate the entire blockchain with modified records, albeit this action would require privileged access and would be detectable through external audit mechanisms. Migration to a distributed blockchain network would provide true decentralization guarantees.

9.1.3 Model Staleness

The XGBoost model is trained on a static snapshot of malware and benign application data. The Android malware landscape is continuously evolving, with new malware families, updated attack techniques, and modified obfuscation strategies emerging regularly. Without periodic retraining on

updated datasets, the model's detection accuracy for newly emerged malware families may gradually decrease over time. The current system supports manual retraining but does not implement automated continuous learning or model drift detection pipelines that would signal when retraining is required.

9.1.4 Dataset Coverage Limitations

The training dataset, while diverse in its inclusion of multiple malware family categories, may not fully represent the complete landscape of Android malware. Malware families that are significantly underrepresented in publicly available datasets — including recently emerged ransomware families and sophisticated nation-state sponsored mobile malware — may exhibit detection accuracy below the system's overall reported performance level. The class imbalance between rare sophisticated malware families and commonly represented malware families is partially addressed through SMOTE augmentation but represents an ongoing challenge.

9.1.5 Performance Under Heavy Concurrent Load

Throughput testing revealed that system performance degrades significantly when concurrent user load exceeds 20 simultaneous users, with scan time increasing by over 150% and error rates rising above 2% at 50 concurrent users. This limitation is attributable to database connection pool exhaustion and the sequential nature of the feature extraction engine under the current single-server deployment architecture. Production deployment at scale would require horizontal scaling architecture, connection pool expansion, and potentially asynchronous feature extraction through task queue systems.

9.2 Scope of Future Work

9.2.1 Hybrid Static-Dynamic Analysis Integration

The most significant enhancement to detection coverage would come from integrating dynamic analysis capabilities alongside the existing static analysis pipeline. A hybrid analysis architecture would execute suspicious applications in an instrumented Android emulator environment when static analysis indicates moderate risk, observing runtime behaviors including system call patterns,

network communication, file system operations, and inter-process communication. Dynamic analysis results would be combined with static features in a unified feature vector for final classification, providing substantially improved coverage for malware that conceals behavior from static examination.

9.2.2 Deep Learning Feature Representation

Future model development could explore replacing or supplementing the hand-engineered feature extraction pipeline with deep learning models that learn representations directly from raw APK data. Convolutional neural networks applied to DEX bytecode sequences have shown promise in the literature for capturing fine-grained structural patterns that manual feature extraction may miss. Graph Neural Networks applied to API call graphs and control flow graphs could capture relational behavioral patterns beyond the scope of flat feature vectors. Transformer architectures adapted for bytecode sequence analysis represent a particularly promising direction given their success in analogous natural language processing tasks.

9.2.3 Federated Learning for Privacy-Preserving Model Updates

Federated learning would enable multiple deployment instances of MalScanX to collectively improve the shared detection model without centralizing sensitive application data at any single server. Each deployment instance would train model updates locally using its analysis data and transmit only the gradient updates — not the raw data — to a central aggregation server. This architecture would enable continuous model improvement from real-world deployment feedback while preserving the privacy of user application data, addressing a significant barrier to large-scale deployment collaboration.

9.2.4 Migration to Distributed Blockchain Network

Replacing the current JSON-based blockchain module with a production-grade distributed blockchain implementation would provide genuine decentralization guarantees that eliminate the trusted administrator dependency in the current implementation. Ethereum smart contracts could automate the model verification and result recording operations with cryptographic enforcement,

enabling fully trustless operation. Hyperledger Fabric, with its permissioned network model and enterprise-grade performance characteristics, may be better suited to the operational requirements of a commercial Android security deployment.

9.2.5 Integration with App Store Platforms

Integration of MalScanX with major application distribution platforms — Google Play Store, Amazon Appstore, Samsung Galaxy Store, and regional Android app markets — would enable automated security screening at the point of submission. API integration with these platforms would require negotiation of data sharing agreements and compliance with platform-specific security and privacy requirements, but would position MalScanX to provide security benefits at the maximum possible scale, screening applications before they reach any user.

9.2.6 Explainable AI and Transparency

The integration of explainable AI techniques — particularly SHAP (SHapley Additive exPlanations) values for per-prediction feature attribution — would enable the system to provide detailed, application-specific explanations for every classification decision. These explanations would assist security researchers in understanding emerging malware techniques, help developers identify and remediate the specific features causing false positive classifications, and improve the transparency and auditability of the detection system for regulatory and compliance purposes.

CHAPTER 10: REFERENCES

- [1] S. Mittal and P. K. G. Pandian, "Deep learning approaches to malware detection and classification," *Int. J. Multidisciplinary Innov. Res. Methodol.*, vol. 3, no. 1, pp. 70–76, 2024.
- [2] H. Yang, Y. Wang, L. Zhang, X. Cheng, and Z. Hu, "A novel Android malware detection method with API semantics extraction," *Comput. Secur.*, vol. 137, Art. no. 103651, Feb. 2024, doi: 10.1016/j.cose.2023.103651.
- [3] R. A. Yunmar, S. S. Kusumawardani, and F. Mohsen, "Hybrid Android malware detection: A review of heuristic-based approach," *IEEE Access*, vol. 12, pp. 41255–41286, 2024, doi: 10.1109/ACCESS.2024.3377658.
- [4] X. Zhang, J. Wang, J. Xu, and C. Gu, "Detection of Android malware based on deep forest and feature enhancement," *IEEE Access*, vol. 11, pp. 29344–29359, 2023, doi: 10.1109/ACCESS.2023.3259301.
- [5] L. Huang and J. Xue, "EAODroid: Android malware detection based on enhanced API order," *Chinese J. Electron.*, vol. 32, no. 3, pp. 1–12, 2023, doi: 10.23919/cje.2021.00.451.
- [6] A. Banik and J. P. Singh, "Android malware detection by correlated real permission couples using FP growth algorithm and neural networks," *IEEE Access*, vol. 11, pp. 112774–112789, 2023, doi: 10.1109/ACCESS.2023.3323845.
- [7] C. S. Yadav et al., "Malware analysis in IoT and Android systems with defensive mechanism," *Electronics*, vol. 11, no. 15, Art. no. 2354, Jul. 2022, doi: 10.3390/electronics11152354.

- [8] A. Mahindru and A. L. Sangal, "Android malware detection using blockchain-based feature validation," *J. Inf. Secur. Appl.*, vol. 68, Art. no. 103248, Aug. 2022, doi: 10.1016/j.jisa.2022.103248.
- [9] P. K. Sharma, N. Kumar, and J. H. Park, "Blockchain-based distributed framework for automotive industry in a smart city," *IEEE Trans. Ind. Inform.*, vol. 18, no. 5, pp. 3099–3109, May 2022, doi: 10.1109/TII.2021.3116095.
- [10] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A. Sadeghi, "DIoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 2023, pp. 756–767, doi: 10.1109/ICDCS.2023.00079.
- [11] Z. Zhu and T. Dumitras, "FeatureSmith: Automatically engineering features for malware detection by mining the security literature," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1284–1298, Mar./Apr. 2023, doi: 10.1109/TDSC.2022.3148591.
- [12] H. Zhou and X. Yang, "An Android malware detection approach based on SIMGRU," *IEEE Access*, vol. 8, pp. 125898–125911, 2020, doi: 10.1109/ACCESS.2020.3007571.

CHAPTER 11: APPENDIX

11.1 Source Code

app.py — Main Flask Application

```
# -----
# MalScanX - Main Flask Application Entry Point
# Project: Holistic Smartphone Data Protection System
# Authors: S.S. Nirmal Raji, S. Lokesh, V. Narendiran
# Guide: Ms. Jaya Sathya, Dept. of Computer Science, BIHER
# -----

from flask import Flask, request, jsonify, render_template, redirect, url_for, session
from flask_mysqldb import MySQL
import os, hashlib, json, time, bcrypt
from xgboost import XGBClassifier
from feature_extractor import FeatureExtractor
from blockchain_module import BlockchainManager
from notification_module import NotificationManager

app = Flask(__name__)
app.secret_key = os.urandom(32)

# MySQL Configuration
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'malscanx_user'
app.config['MYSQL_PASSWORD'] = 'secure_password_here'
app.config['MYSQL_DB'] = 'malscanx_db'
mysql = MySQL(app)

# Initialize core modules
blockchain = BlockchainManager()
notifier = NotificationManager()
extractor = FeatureExtractor()
model = XGBClassifier()
model.load_model('models/malscanx_v1.json')

RISK_LEVELS = {
    'SAFE': (0.00, 0.50),
    'LOW': (0.50, 0.60),
    'MODERATE': (0.60, 0.80),
    'HIGH': (0.80, 0.95),
    'CRITICAL': (0.95, 1.00)
}

def assign_risk_level(prediction, confidence):
    if prediction == 0:
```

```

        return 'SAFE'
    for level, (lo, hi) in RISK_LEVELS.items():
        if level == 'SAFE': continue
        if lo <= confidence < hi:
            return level
    return 'CRITICAL'

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password'].encode('utf-8')
        cursor = mysql.connection.cursor()
        cursor.execute('SELECT id, role, password_hash FROM users WHERE username=%s',
            (username,))
        user = cursor.fetchone()
        if user and bcrypt.checkpw(password, user[2].encode('utf-8')):
            session['user_id'] = user[0]
            session['role'] = user[1]
            return redirect(url_for('dashboard'))
        return render_template('login.html', error='Invalid credentials')
    return render_template('login.html')

@app.route('/scan', methods=['POST'])
def scan_apk():
    if 'user_id' not in session:
        return jsonify({'error': 'Authentication required'}), 401

    apk_file = request.files.get('apk')
    if not apk_file:
        return jsonify({'error': 'No APK file provided'}), 400

    # Step 1: Verify blockchain model integrity
    stored_hash = blockchain.get_model_hash()
    active_hash = blockchain.compute_hash({'model':
open('models/malscanx_v1.json', 'rb').read().hex()})
    if stored_hash != active_hash:
        notifier.send_admin_alert('MODEL_INTEGRITY_FAILURE', {'timestamp': time.time()})
        return jsonify({'error': 'Model integrity verification failed'}), 500

    # Step 2: Save APK temporarily and compute APK hash
    apk_path = f'/tmp/{hashlib.md5(apk_file.filename.encode()).hexdigest()}.apk'
    apk_file.save(apk_path)
    with open(apk_path, 'rb') as f:
        apk_hash = hashlib.sha256(f.read()).hexdigest()

```

```

# Step 3: Check blockchain for previous scan of same APK
cached = blockchain.lookup_result(apk_hash)
if cached:
    os.remove(apk_path)
    return jsonify(**cached, 'source': 'blockchain_cache'})

# Step 4: Extract features
features = extractor.extract_all(apk_path)

# Step 5: Classify
prediction = int(model.predict([features])[0])
confidence = float(max(model.predict_proba([features])[0]))
risk_level = assign_risk_level(prediction, confidence)

result = {
    'apk_hash': apk_hash,
    'app_name': apk_file.filename,
    'prediction': 'MALICIOUS' if prediction == 1 else 'BENIGN',
    'confidence': round(confidence, 4),
    'risk_level': risk_level,
    'timestamp': time.time(),
    'model_ver': 'MalScanX_v1.0'
}

# Step 6: Record result on blockchain
blockchain.record_result(result)

# Step 7: Enforce blocking and send notifications
if prediction == 1 and risk_level in ('HIGH', 'CRITICAL'):
    notifier.send_alert(session['user_id'], risk_level, apk_file.filename)

# Step 8: Persist to MySQL
cursor = mysql.connection.cursor()
cursor.execute(
    'INSERT INTO scan_results (user_id, apk_hash, app_name, prediction, confidence,
risk_level)
    VALUES (%s, %s, %s, %s, %s, %s)',
    (session['user_id'], apk_hash, apk_file.filename, prediction, confidence,
risk_level)
)
mysql.connection.commit()
os.remove(apk_path)
return jsonify(result)

if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0', port=5000)
    
```

blockchain_module.py — Blockchain Manager

```
import hashlib, json, time

class BlockchainManager:
    """JSON-based hash-chain blockchain for model and result verification."""

    def __init__(self, chain_file='blockchain/malscanx_chain.json'):
        self.chain_file = chain_file
        self.model_hash = None
        self.chain = []
        self._load_or_init()

    def _load_or_init(self):
        try:
            with open(self.chain_file, 'r') as f:
                data = json.load(f)
                self.chain = data['chain']
                self.model_hash = data.get('model_hash')
        except FileNotFoundError:
            self._create_genesis_block()

    def _create_genesis_block(self):
        block = {'index': 0, 'timestamp': time.time(),
                'data': 'MalScanX Genesis Block',
                'prev_hash': '0' * 64}
        block['hash'] = self.compute_hash(block)
        self.chain.append(block)
        self._persist()

    def compute_hash(self, obj):
        serialized = json.dumps(obj, sort_keys=True, default=str).encode()
        return hashlib.sha256(serialized).hexdigest()

    def add_block(self, data):
        last = self.chain[-1]
        block = {'index': len(self.chain), 'timestamp': time.time(),
                'data': data, 'prev_hash': last['hash']}
        block['hash'] = self.compute_hash(block)
        self.chain.append(block)
        self._persist()
        return block

    def store_model_hash(self, model_path, version):
        with open(model_path, 'rb') as f:
            raw = f.read()
        h = hashlib.sha256(raw).hexdigest()
        self.model_hash = h
        self.add_block({'type': 'MODEL_STORE', 'version': version,
                        'hash': h, 'stored_at': time.time()})
        return h
```

```

def get_model_hash(self):
    return self.model_hash

def record_result(self, result):
    self.add_block({'type': 'SCAN_RESULT', 'data': result})

def lookup_result(self, apk_hash):
    for block in reversed(self.chain):
        d = block.get('data', {})
        if isinstance(d, dict) and d.get('type') == 'SCAN_RESULT':
            if d.get('data', {}).get('apk_hash') == apk_hash:
                return d['data']
    return None

def verify_chain_integrity(self):
    for i in range(1, len(self.chain)):
        curr, prev = self.chain[i], self.chain[i-1]
        prev_copy = {k: v for k, v in prev.items() if k != 'hash'}
        if prev['hash'] != self.compute_hash(prev_copy):
            raise ValueError(f'Block {i-1} hash mismatch')
        if curr['prev_hash'] != prev['hash']:
            raise ValueError(f'Block {i} chain linkage broken')
    return True

def _persist(self):
    import os
    os.makedirs(os.path.dirname(self.chain_file), exist_ok=True)
    with open(self.chain_file, 'w') as f:
        json.dump({'chain': self.chain, 'model_hash': self.model_hash}, f, indent=2)
    
```

feature_extractor.py — Static Feature Extraction Engine

```

import numpy as np
from androguard.misc import AnalyzeAPK

PERMISSIONS_LIST = [ # 130 security-relevant permissions (abbreviated)
    'READ_SMS', 'SEND_SMS', 'RECEIVE_SMS', 'READ_CONTACTS', 'WRITE_CONTACTS',
    'ACCESS_FINE_LOCATION', 'ACCESS_COARSE_LOCATION', 'RECORD_AUDIO',
    'CAMERA', 'READ_CALL_LOG', 'WRITE_CALL_LOG', 'INTERNET', 'RECEIVE_BOOT_COMPLETED',
    'READ_PHONE_STATE', 'CALL_PHONE', 'CHANGE_WIFI_STATE', 'BLUETOOTH',
    # ... 113 additional permissions ...
]

SENSITIVE_APIS = [ # 30 security-relevant Android API calls
    'java/lang/Runtime;->exec', 'android/telephony/SmsManager;->sendTextMessage',
    'java/lang/reflect/Method;->invoke', 'java/net/Socket;-><init>',
    'javax/crypto/Cipher;->getInstance',
    'android/location/LocationManager;->requestUpdates',
    
```

```

# ... 24 additional API patterns ...
]

INTENT_FILTERS = [ # 10 high-significance intent actions
    'android.intent.action.BOOT_COMPLETED',
    'android.intent.action.SMS_RECEIVED',
    'android.telephony.action.CARRIER_CONFIG_CHANGED',
    # ... 7 additional intent filters ...
]

OPCODE_GROUPS = { # 10 opcode behavioral categories
    'invoke_virtual': ['invoke-virtual'],
    'invoke_static': ['invoke-static'],
    'invoke_direct': ['invoke-direct'],
    'const_string': ['const-string', 'const-string/jumbo'],
    'new_instance': ['new-instance'],
    'array_ops': ['new-array', 'aget', 'aput', 'fill-array-data'],
    'field_access': ['iget', 'iput', 'sget', 'sput'],
    'branch_ops': ['if-eq', 'if-ne', 'if-lt', 'if-ge', 'if-gt', 'if-le'],
    'move_ops': ['move', 'move-result', 'move-object'],
    'return_ops': ['return', 'return-object', 'return-wide']
}

class FeatureExtractor:
    def extract_all(self, apk_path):
        a, d, dx = AnalyzeAPK(apk_path)
        features = []
        features.extend(self._permissions(a)) # 130 features
        features.extend(self._components(a)) # 20 features
        features.extend(self._api_calls(dx)) # 30 features
        features.extend(self._intents(a)) # 10 features
        features.extend(self._opcodes(d)) # 10 features
        return np.array(features, dtype=np.float32) # 200 features total

    def _permissions(self, a):
        declared = set(a.get_permissions())
        return [1.0 if p in declared else 0.0 for p in PERMISSIONS_LIST]

    def _components(self, a):
        return [
            len(a.get_activities()), # activity count
            len(a.get_services()), # service count
            len(a.get_receivers()), # receiver count
            len(a.get_providers()), # provider count
            float(len(a.get_activities()) > 5), # high activity flag
            # ... 15 additional component features ...
        ]

    def _api_calls(self, dx):
        called = set()
        for cls in dx.get_classes():

```

```

        for meth in cls.get_methods():
            for _, call, _ in meth.get_xref_to():
                called.add(f'{call.class_name}->{call.name}')
    return [1.0 if any(api in c for c in called) else 0.0 for api in SENSITIVE_APIS]

def _intents(self, a):
    declared = set()
    for a_name in a.get_activities():
        for intent in (a.get_intent_filters('activity', a_name) or []):
            declared.update(intent.get('action', []))
    return [1.0 if i in declared else 0.0 for i in INTENT_FILTERS]

def _opcodes(self, d):
    counts = {g: 0 for g in OPCODE_GROUPS}
    total = 0
    for dex in d:
        for cls in dex.get_classes():
            for meth in cls.get_methods():
                for ins in meth.get_instructions():
                    op = ins.get_name().lower()
                    total += 1
                    for grp, ops in OPCODE_GROUPS.items():
                        if op in ops:
                            counts[grp] += 1
    if total == 0: return [0.0] * len(OPCODE_GROUPS)
    return [counts[g] / total for g in OPCODE_GROUPS]

```

11.2 Output Screenshots

The following section describes the interface screens implemented in the MalScanX system, corresponding to each page in the system implementation workflow. The descriptions below map to the output pages referenced in Chapter 5.

Figure 11.1 — Android Malware Detection Main Page

The main landing page presents users with the MalScanX branding, a concise description of the system's purpose and capabilities, and three prominent navigation buttons directing users to their respective role entry points: Admin Login, Developer Registration, and User Login. The page uses a professional blue-and-white color scheme consistent with the Bootstrap 5 theme applied throughout the system. A brief animated statistics banner displays system-level metrics including total applications scanned, percentage of malicious applications detected, and blockchain verification status.

Figure 11.2 — Admin Login Page

The administrator authentication page presents a clean login form with username and password fields, a bcrypt-protected authentication workflow, and a CAPTCHA verification element for bot protection. Upon successful authentication with valid administrator credentials, the user is redirected to the comprehensive administration dashboard. Invalid credentials are handled with a clear, non-specific error message to prevent username enumeration attacks. The login form includes a secure password visibility toggle and a password reset workflow accessible through the institutional IT support contact.

Figure 11.3 — Upload App Dataset Page

The dataset upload interface presents administrators with a drag-and-drop file upload area supporting batch APK import, a format selection dropdown for supported dataset types (raw APK collection, CSV feature file, hybrid format), a label configuration panel for specifying the mapping between file naming conventions and malware/benign class labels, and a dataset validation preview that displays class distribution statistics and sample count before confirming the import operation. Progress bars indicate upload and processing status for large dataset imports.

Figure 11.4 — Preprocessing Page

The preprocessing interface displays a step-by-step visual summary of the data transformation pipeline applied to the imported dataset. Visual elements include a before/after class distribution comparison showing the effect of SMOTE augmentation and undersampling, a missing values summary table with imputation decisions, a feature normalization range display showing the min-max parameters computed from the training set, and a final dataset summary confirming the training/validation/test split proportions and sample counts.

Figure 11.5 — Feature Extraction Page

The feature extraction results page provides administrators with a comprehensive view of the features extracted from dataset applications. A tabbed interface presents results organized by feature

category: permission heatmap showing the distribution of permission declarations across benign and malicious samples, component count distribution charts, API call frequency comparison between classes, intent filter presence comparison, and opcode distribution box plots. Export functionality allows administrators to download the full extracted feature matrix in CSV format for external analysis.

Figure 11.6 — Correlation Graph Page

The correlation analysis page displays a color-coded correlation heatmap computed over the 200-feature training matrix. Highly correlated feature pairs are highlighted to guide feature selection decisions. The page also presents the top 20 most discriminative features ranked by their mutual information with the class label, enabling administrators to assess the quality and informativeness of the current feature set.

Figure 11.7 — XGBoost Classification Output Page

The model training results page displays the trained XGBoost model's performance on the validation and test sets. Visual elements include a confusion matrix with color-coded cells, precision-recall curves for both classes, an ROC curve with AUC annotation, and a feature importance bar chart showing the top 20 most influential features. Training history charts display loss and accuracy progression across boosting rounds, with the early stopping point marked clearly.

Figure 11.8 — AndroMal Model Build Page

The model build status page provides a complete record of the completed training process including the hyperparameter configuration used, training duration, final performance metrics, blockchain storage confirmation with the model hash value and block index, and model version information. Administrators can initiate model retraining from this page with optional hyperparameter configuration override.

Figure 11.9 — Developer Registration Page

The developer registration interface collects developer account information including display name, email address, organization affiliation, and secure password. Email verification is required to activate the account, preventing fake account creation. Registered developers gain access to the application submission portal and can manage multiple submitted applications through a personal dashboard.

Figure 11.10 — Developer Login Page

The developer authentication page provides secure login functionality for registered application developers. Two-factor authentication is available as an optional security enhancement. Successful authentication redirects developers to their personal dashboard showing submission history, scan results, and blockchain verification certificates for all submitted applications.

Figure 11.11 — Add New App Page

The application submission portal allows developers to upload APK files for security screening. The form accepts the APK file, application display name, version number, and optional description. A real-time file validation indicator confirms that the uploaded file is a valid APK before submission. After submission, the developer is redirected to a waiting page that displays live analysis progress updates and redirects automatically to the results page upon completion.

Figure 11.12 — App View Page

The detailed scan results page presents the complete analysis outcome for a submitted application. Information displayed includes the overall verdict (SAFE/MALICIOUS) in a prominent color-coded banner, the assigned risk level, the confidence score, a list of the top feature indicators that contributed to the classification, the blockchain transaction reference number and block index, and the timestamp of the analysis. Developers whose applications receive false positive classifications can submit a dispute request through a button on this page.

Figure 11.13 — User Registration and User Login Pages

The end user registration and authentication pages provide simplified account creation and login workflows appropriate for non-technical users. Registration requires only a display name, email, and password. The login interface uses the same Bootstrap card design as the other authentication pages for visual consistency. Password strength feedback is provided during registration, and account recovery is available through email-based password reset.

Figure 11.14 — Apps List and Apps Search Pages

The application browsing interface presents users with a searchable, filterable catalog of applications in the MalScanX database. Each application entry displays a safety status badge, application name, version, and analysis date. Search functionality supports filtering by application name, safety status, and date range. Users can click any entry to view the full analysis report and blockchain verification record.

Figure 11.15 — Block Information Pages

The blockchain transparency pages provide direct visibility into the blockchain records maintained by MalScanX. Two views are available: a chain overview showing all blocks with their index, timestamp, data type, and hash preview, and a block detail view showing the complete JSON content of any selected block. The chain integrity verification status is displayed prominently at the top of the page, with a green indicator confirming that all hash linkages are valid. Users and administrators can verify that their scan results are correctly recorded by searching for the specific transaction hash provided in their scan result certificate.

END OF PROJECT DOCUMENTATION

Department of Computer Science
Bharath Institute of Higher Education and Research
(Deemed to be University — Accredited by NAAC)

Chennai — April 2025